ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN CUỐI KỲ

MÔN HỌC: CE118 - THIẾT KẾ LUẬN LÝ SỐ

Lớp: CE118.Q14

ĐỀ TÀI: THIẾT KẾ MẠCH ASIC TÍNH TOÁN HÀM SIN VÀ COS FIXED POINT 16-BIT DỰA TRÊN THUẬT TOÁN CORDIC KẾT HỢP KĨ THUẬT TỐI ƯU PIPELINE

Giảng viên hướng dẫn: **ThS. Hồ Ngọc Diễm**

Sinh viên thực hiện:

1. **Lương Cao Pháp** **24521288**
2. **Đỗ Khải** **24520720**

TP. HỒ CHÍ MINH – Tháng 12 năm 2025

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc đến cô Hồ Ngọc Diễm – giảng viên môn Thiết kế luận lý số (CE118.Q14).

Trong suốt quá trình học tập và thực hiện đồ án môn học, chúng em luôn nhận được sự quan tâm, hướng dẫn tận tình cùng những góp ý quý báu từ cô. Chính sự chỉ dẫn chi tiết, những nhận xét sâu sắc và sự kiên nhẫn của cô đã giúp chúng em từng bước hoàn thiện kiến thức và hoàn thành đồ án một cách tốt nhất.

Mặc dù đã rất cố gắng, nhưng do thời gian và năng lực còn hạn chế, đồ án của chúng em khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự cảm thông và những ý kiến đóng góp từ cô để đồ án được hoàn thiện hơn.

Cuối cùng, chúng em xin kính chúc cô luôn dồi dào sức khỏe, nhiều niềm vui trong công việc và tiếp tục truyền đạt những kiến thức quý báu cho các thế hệ sinh viên sau này.

Chúng em xin chân thành cảm ơn!

Thành phố Hồ Chí Minh, tháng 12 năm 2025

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU 2

1.1. Đặt vấn đề 2

1.2. Mục tiêu đề tài 3

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 4

2.1. Thuật toán tìm kiếm nhị phân 4

2.2. Thuật toán CORDIC 5

2.2.1. Định nghĩa các hàm lượng giác sin và cos 5

2.2.2. Phép xoay vector thật (Real Rotation) 6

2.2.3. Phép xoay vector giả (Pseudo Rotation) 7

2.2.4. Ý tưởng thuật toán CORDIC tính sin và cos 8

2.2.5. Hệ số tỷ lệ scale K 9

2.2.6. Bảng tra cứu góc arctan $\mathbf{2-i}$ LUT (Lookup Table) 10

2.2.7. Giới hạn về phạm vi góc tính và giải pháp 10

2.3. Số Fixed Point 12

2.3.1. Biểu diễn số thực bằng Fixed Point 12

2.3.2. Phép toán số học với Fixed Point 13

2.3.3. Xác định tầm biểu diễn của số Fixed Point 16 bit vào mạch thiết kế 14

CHƯƠNG 3. THIẾT KẾ MẠCH TÍNH SIN VÀ COS 14

3.1. Mã giả (Pseudo-code) 14

3.2. Xây dựng Datapath đơn giản 16

3.3. Xây dựng sơ đồ ASM 17

3.4. Tối ưu phần cứng với Register Sharing 19

3.5. Tối ưu phần cứng với Bus Sharing 20

3.6. Thiết kế các thành phần của Datapath 24

`

3.6.1. Thanh ghi (Register) 16 bit 24
3.6.2. Khối AU 26
3.6.3. Bộ dịch phải n bit 29
3.6.4. ROM (Read Only Memory) 30
3.6.5. Bộ đếm 33
3.6.6. Các giá trị hằng số 34
3.6.7. Hoàn thiện khối Datapath 36
3.7. Thiết kế khối Controller 37
3.7.1. Thiết kế máy trạng thái FSM 37
3.7.2. Thiết kế các tín hiệu điều khiển 38
3.8. Hoàn thiện mạch ASIC 43
3.8.1. Mạch so sánh điều kiện dừng 43
3.8.2. Hoàn thiện mạch ASIC 44
3.9. Mô phỏng mạch trên Quartus 45
3.10. Đánh giá chi phí và hiệu suất 46
3.10.1. Chi phí (Cost) 46
3.10.2. Tần số Clock tối đa (F max) 46
3.10.3. Số chu kỳ thực thi 47
CHƯƠNG 4. TỐI ƯU MẠCH VÀ ĐÁNH GIÁ 47
4.1. Phân tích đường đi lớn nhất (Critical Path) 47
4.2. Tối ưu mạch bằng pipeline khối chức năng 48
4.2.1. Multi-cycling và pipeline 2 tầng khối AU 48
4.2.2. Thiết kế khối AU 2 tầng 50
4.2.3. Thiết kế máy trạng thái FSM cho mạch sau khi pipeline AU 2 tầng 51
4.2.4. Thiết kế các tín hiệu điều khiển cho mạch sau khi pipeline AU 2 tầng 53
4.2.5. Đánh giá hiệu suất và so sánh mạch sau khi pipeline AU 2 tầng 55
4.3. Tối ưu mạch bằng pipeline datapath và AU 2 tầng 56

4.3.1. Pipeline Datapath 2 tầng 56
4.3.2. Sơ đồ pipeline Datapath 2 tầng và AU 2 tầng..... 56
4.4. Đánh giá thông số các mạch..... 58
CHƯƠNG 5. HIỆN THỰC MẠCH TRÊN KIT DE2 59
5.1. Hiện thị số fixed-point 16 bit trên LED 7 đoạn 59
5.2. Thuật toán Double Dabble 59
5.3. Hiện thực kết quả mạch thiết kế trên KIT DE2 60
5.3.1. Link video nạp mạch ASIC tính sin và cos trên KIT DE2 60
5.3.2. Kịch bản nạp KIT..... 61
5.3.3. Kết quả nạp mạch không sử dụng pipeline trên KIT DE2..... 61

DANH SÁCH HÌNH ẢNH

Hình 2.1. Định nghĩa lượng giác $cos\alpha$ và $sin\alpha$ trên đường tròn lượng giác 5
Hình 2.2. Phép xoay vector thật 6
Hình 2.3. Phép xoay vector giả 8
Hình 2.4. Chuỗi xoay giả từ vector gốc 9
Hình 2.5. Phạm vi góc xoay của thuật toán CORDIC 11
Hình 2.6. Biến đổi vector ở góc phần tư thứ II và thứ IV về vector ở góc phần tư thứ I và IV. 11
Hình 2.7. Biểu diễn fixed point không dấu 12
Hình 2.8. Biểu diễn fixed point có dấu 12
Hình 3.1. Mã giả giải thuật tính sin và cos dựa trên thuật toán CORDIC 16
Hình 3.2. Sơ đồ ASM 18
Hình 3.3. Đánh nhãn cho các đường kết nối dữ liệu 21
Hình 3.4. Datapath sau khi đã tối ưu bằng kĩ thuật Bus Sharing 24
Hình 3.5Đóng gói mạch cộng trừ 16 bit 28
Hình 3.6. Ý tưởng thiết kế bộ dịch nhị phân 29
Hình 3.7. Vẽ mạch cho bộ dịch phải n bit 30
Hình 3.8. Đóng gói bộ dịch phải n bit 30
Hình 3.9. Ý tưởng thiết kế ROM từ decoder và các cổng logic OR 32
Hình 3.10. Mã verilog thiết kế khối ROM 33
Hình 3.11. Đóng gói khối ROM 33
Hình 3.12. Thiết kế mạch cho bộ đếm lên 4 bit 34
Hình 3.13. Đóng gói mạch cho bộ đếm lên 4 bit 34
Hình 3.14. Vẽ mạch cho giá trị hằng K 35
Hình 3.15. Đóng gói khối giá trị hằng K 35
Hình 3.16. Vẽ mạch cho giá trị hằng 0 35
Hình 3.17. Đóng gói khối giá trị hằng 0 35
Hình 3.18. Vẽ mạch Datapath hoàn thiện trên mô phỏng Quartus 36
Hình 3.19. Đóng gói Datapath 36
Hình 3.20. Thiết kế mạch chuyển trạng thái 38
Hình 3.21. Vẽ mạch cho các tín hiệu điều khiển của khối Controller 42
Hình 3.22. Đóng gói khối Controller 43
Hình 3.23. Vẽ mạch so sánh với giá trị 13 44
Hình 3.24. Đóng gói khối so sánh với giá trị 13 44

Hình 3.25. Hoàn thiện mạch ASIC tính sin và cos của góc ban đầu 44
Hình 3.26. Đóng gói mạch ASIC tính sin và cos của góc ban đầu 44
Hình 3.27. Mạch ASIC tính toán sin và cos 45
Hình 3.28. Kết quả mô phỏng trên Quartus 45
Hình 3.29. Fmax của mạch trên mô phỏng Quartus 47
Hình 4.1. Thời gian hoạt động của các khối chức năng ở các trạng thái 48
Hình 4.2. Đường đi critical path khi thực hiện 2 phép tính trên 1 khối AU đơn chu kì 48
Hình 4.3. Multicycling khối AU 49
Hình 4.4. Multicycling và Pipeline 2 tầng khối AU 49
Hình 4.5. Thiết kế pipeline 2 tầng khối AU 50
Hình 4.6. Thêm các Flip Flop ở giữa các vị trí khi pipeline 2 tầng khối AU 51
Hình 4.7. Fmax của mạch sau khi áp dụng AU 2 tầng 55
Hình 4.8. Datapath sau khi thực hiện pipeline Datapath 2 tầng và khối AU 2 tầng 57
Hình 4.9. Vẽ mạch sau khi cải tiến pipeline Datapath 2 tầng và khối AU 2 tầng 58
Hình 5.1. Thuật toán Double Dabble chuyển đổi số nhị phân 16 bit sang số BCD 20 bit. 60
Hình 5.2. Vẽ mạch tổ hợp theo thuật toán Double Dabble 60
Hình 5.3. Giá trị cos của góc $\pi/3$ trên mô phỏng KIT 61
Hình 5.4. Giá trị sin của góc $\pi/3$ trên mô phỏng KIT 62
Hình 5.5. Giá trị cos của góc $-\pi/4$ trên mô phỏng KIT 62
Hình 5.6. Giá trị sin của góc $-\pi/4$ trên mô phỏng KIT 63

`

DANH SÁCH BẢNG

Bảng 2.1. Bảng LUT các giá trị góc $arctan2 - i$ 10
Bảng 3.1. Xây dựng Datapath đơn giản 17
Bảng 3.2. Bảng thời gian sống các biến 20
Bảng 3.3. Bảng sử dụng các kết nối Input tại mỗi trạng thái (Connectivity usage) 22
Bảng 3.4. Bảng sử dụng các kết nối Output tại mỗi trạng thái (Connectivity usage) 22
Bảng 3.5. Bảng chức năng của thanh ghi 25
Bảng 3.6. Vẽ mạch thanh ghi 16 bit 26
Bảng 3.7. Đóng gói thanh ghi 16 bit 26
Bảng 3.8. Bảng chức năng của khối AU 26
Bảng 3.9. Mạch cộng sử dụng Full Adder 27
Bảng 3.10. Mạch trừ sử dụng Full Adder 27
Bảng 3.11. Vẽ mạch bộ cộng 8 bit 28
Bảng 3.12. Vẽ mạch cộng trừ 16 bit được ghép từ 2 bộ cộng trừ 8 bit 28
Bảng 3.13. Bảng lưu giá trị các góc xoay giả $arctan2 - i$ 31
Bảng 3.14. Bảng chức năng của bộ đếm 34
Bảng 3.15. Bảng chuyển trạng thái của FSM 37
Bảng 3.16. Bảng hành động của Datapath 39
Bảng 3.17. Bảng sự thật các tín hiệu ngõ ra điều khiển của khối Controller 40
Bảng 4.1. Bảng thời gian thực thi của mạch với khối AU 2 tầng 52
Bảng 4.2. Bảng chuyển trạng thái khi kết hợp khối AU 2 tầng 53
Bảng 4.3. Bảng sự thật các tín hiệu điều khiển của khối Controller khi áp dụng AU 2 tầng 54
Bảng 4.4. So sánh hiệu suất trước và sau khi tối ưu pipeline khối AU 2 tầng 55
Bảng 4.5. Sơ đồ thời gian của mạch khi áp dụng pipeline 2 tầng Datapath kết hợp với khối AU 2 tầng 56
Bảng 4.6. Bảng so sánh trước và sau khi áp dụng các kỹ thuật tối ưu pipeline khối AU 2 tầng và Datapath 2 tầng. 58
Bảng 5.1. Bảng các giá trị góc z để mô phỏng nạp KIT DE2 61

TÓM TẮT ĐỒ ÁN

Trong đồ án này, chúng em xin trình bày việc thiết kế một mạch chức năng ASIC có thể thực hiện được chức năng tính toán hàm sin và cos trên số fixed point 16-bit dựa trên thuật toán CORDIC với phạm vi tính toán từ góc -π tới π một cách đơn giản. Trên cơ sở mạch ban đầu, chúng em tiến hành nâng cao hiệu suất hoạt động của mạch bằng các phương pháp chia sẻ phần cứng (hardware sharing) và tối ưu hiệu năng bằng kĩ thuật pipeline. Việc tối ưu chủ yếu thực hiện pipeline khối chức năng (functional unit) và đường dữ liệu (datapath).

Nội dung bao gồm 4 phần chính:

- Cơ sở lý thuyết: giới thiệu lý thuyết toán học liên quan đến bài toán, chứng minh giải thuật CORDIC, điểm mạnh và những giới hạn khi sử dụng để áp dụng ý tưởng để hiện thực mạch.
- Xây dựng và thực thi mạch: xác định thuật toán, mô tả các thành phần của mạch, xây dựng sơ đồ ASM, hiện thực khối Datapath và Control.
- Tối ưu hiệu suất mạch và so sánh: phân tích những hạn chế của mạch ban đầu, sau đó sử dụng các kĩ thuật tối ưu phần cứng và hiệu suất để giảm độ trễ xử lý bằng pipeline. Sau đó phân tích và so sánh mạch trước và sau khi tối ưu về các tiêu chí logic cost, tần số tối đa của mạch, số chu kỳ clock dựa trên mô phỏng Quartus.
- Nạp mạch mô phỏng trên KIT DE2: tiến hành nạp trên KIT DE2 để chạy mô phỏng kết quả, đồng thời giải quyết bài toán hiện thị số fixed point 16-bit lên trên LED 7 đoạn của KIT DE2.

`

CHƯƠNG 1. GIỚI THIỆU

1.1. Đặt vấn đề

Trong nhiều lĩnh vực kỹ thuật hiện đại ngày nay như điện - điện tử, tự động hóa, xử lý tín hiệu số và điều khiển tự các động, ngoài các phép toán học thông thường thì không thể tránh khỏi việc phải tính toán các phép toán lượng giác, đặc biệt là tính giá trị sin và cos. Các giá trị này giữ vai trò then chốt trong việc mô tả, phân tích và điều khiển của toàn bộ hệ thống tiêu biểu như điều khiển động cơ, xử lý tín hiệu, đồ họa máy tính, …

Trong toán học, có một số cách khai triển công thức tính giá trị sin(x), cos(x) chẳng hạn như chuỗi Maclaurin :

$$\cos x = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k)!} x^{2k} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$$

$$\sin x = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!} x^{2k+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

Tuy nhiên việc tính toán các hàm lượng giác bằng các phương pháp truyền thống bằng các định nghĩa như trên thường yêu cầu nhiều phép nhân, tính giai thừa, phép chia và các khối phần cứng phức tạp dẫn đến khó triển khai trên các hệ thống phần cứng chuyên dụng. Điều này đặc biệt bất lợi trong thiết kế mạch tích hợp chuyên dụng (ASIC), vi điều khiển, FPGA và các mạch số chuyên dụng yêu cầu cao về tốc độ, độ chính xác và tài nguyên.

Thuật toán CORDIC (COordinate Rotaion DIgital Computer) ra đời nhằm giải quyết hiệu quả bài toán trên. CORDIC cho phép tính toán các hàm lượng giác như sin và cos chỉ bằng các phép dịch bit, cộng và trừ, rất phù hợp đề triển khai trên phần cứng số. Như đặc điểm này, CORDIC đã trở thành một thuật toán quan trọng trong thiết kế hệ thống số.

`

1.2. Mục tiêu đề tài

Thiết kế được mạch ASIC thực hiện tính toán các hàm lượng giác sin và cos trên số fixed-point 16-bit trong phạm vi góc từ -π đến π, dựa trên thuật toán CORDIC.

Mô tả và trình bày chi tiết các bước thiết kế phần cứng bao gồm Datapath và Control với sơ đồ ASM.

Nâng cao hiệu suất mạch bằng các kỹ thuật tối ưu phần cứng và tối ưu hiệu năng bằng pipeline.

Phân tích, đánh giá và so sánh hiệu quả của mạch trước và sau khi tối ưu về các tiêu chí như logic cost, tần số hoạt động tối đa, số chu kỳ clock trên mô phỏng Quartus.

Kiểm chứng mạch trên KIT DE2, bao gồm nạp mạch, mô phỏng kết quả và hiển thị giá trị số fixed-point 16-bit trên LED 7 đoạn.

`

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Thuật toán tìm kiếm nhị phân

Trước khi nghiên cứu thuật toán CORDIC, chúng ta cùng tìm hiểu về thuật toán tìm kiếm nhị phân để từ đó dễ dàng tiếp cận và hiểu rõ ý tưởng cốt lõi của thuật toán thông qua bài toán như sau:

Cho một đoạn thẳng có chiều dài cố định được đánh số từ 0 đến 100. Ta đặt một điểm mục tiêu ở tại một ví trí trên đoạn thẳng này, ta biết vị trí chính xác của mục tiêu trên đoạn thẳng này đang ở tọa độ là 71,4.

Ta chỉ được phép:

- Đi sang trái hoặc sang phải
- Mỗi lần đi được một khoảng cách cố định
- Khoảng cách cố định của bước sau sẽ bằng một nửa khoảng cách cố định của bước trước

Ban đầu bạn đang đang ở vị trí là 0. Khoảng cách cố định bạn có thể bước ở bước đầu tiên là 50. Hãy tìm cách đi tới vị trí của mục tiêu.

Bắt đầu từ vị trí ban đầu là 0. Ta quan sát thấy điểm mục tiêu nằm tại vị trí 71,4 nên ta di chuyển sang phải đến vị trí 50.

Tại vị trí 50, mục tiêu vẫn nằm bên phải vị trí hiện tại, trong khi bước di chuyển lúc này chỉ còn 25, nên ta tiếp tục bước sang phải đến vị trí 75.

Đến vị trí 75, mục tiêu lại nằm bên trái, và bước di chuyển giảm còn 12,5. Do đó, ta bước sang trái đến vị trí 67,5.

Quá trình trên được lặp lại với độ dài bước giảm dần, giúp vị trí hiện tại ngày càng tiến gần đến vị trí mục tiêu.

Tương tự, thuật toán CORDIC cũng dựa trên tư duy chia nhỏ bài toán và tiệm cận đến giá trị qua từng bước lặp. Thay vì tìm kiếm một phần tử trong tập dữ liệu, CORDIC thực hiện việc xác định góc xoay bằng cách liên tục xoay vector theo các góc ngày càng nhỏ trên đường tròn đơn vị. Ở mỗi bước lặp, thuật toán chỉ cần đưa ra quyết định về chiều xoay (xoay thuận hoặc xoay ngược), tương đương với việc chọn nửa khoảng phù hợp trong tìm kiếm nhị phân.

`

2.2. Thuật toán CORDIC

2.2.1. Định nghĩa các hàm lượng giác sin và cos

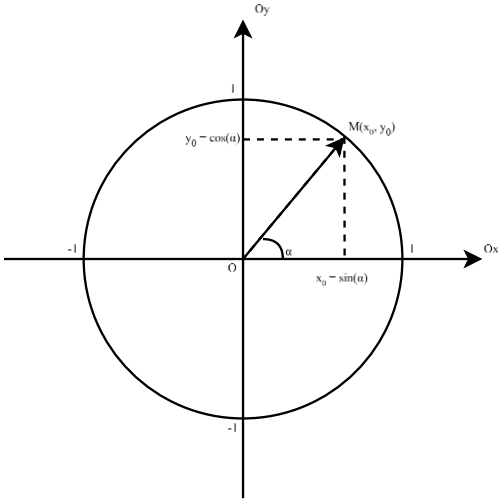
Trong toán học nói chung và lượng giác học nói riêng, các hàm lượng giác là các hàm toán học của góc, được dùng khi nghiên cứu tam giác và các hiện tượng có tính chất tuần hoàn. Các hàm lượng giác của một góc thường được định nghĩa bởi tỷ lệ chiều dài hai cạnh của tam giác vuông chứa góc đó, hoặc tỷ lệ chiều dài giữa các đoạn thẳng nối các điểm đặc biệt trên vòng tròn đơn vị.

Những định nghĩa hiện đại hơn thường coi các hàm lượng giác là chuỗi số vô hạn hoặc là nghiệm của một số phương trình vi phân, điều này cho phép hàm lượng giác có thể có đối số là một số thực hay một số phức bất kì. Chẳng hạng như ta có thể dùng chuỗi Taylor để phân tích hàm sin và cos, mọi góc của hàm đo bằng giá trị radian thực:

$$\cos x = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k)!} x^{2k} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$$

$$\sin x = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!} x^{2k+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

Tuy nhiên, trong phạm vi nghiên cứu của đồ án này, ta sẽ định nghĩa các hàm lượng giác dựa trên đường tròn đơn vị. Cho vòng tròn lượng giác (vòng tròn đơn vị) có bán kính bằng 1 cố định, có tâm tại gốc tọa độ $(0,0)$ trên hệ tọa độ Oxy. Lấy một điểm $M(x_0, y_0)$ bất kì nằm trên đường tròn đơn vị. là Khi đó vector $\overrightarrow{OM}$ tạo với trục Ox một góc α.



Hình 2.1. Định nghĩa lượng giác $cos(\alpha)$ và $sin(\alpha)$ trên đường tròn lượng giác

Giá trị $\cos(\alpha)$ và $\sin(\alpha)$ lần lượt chính là hoành độ và tung độ của điểm M, tức là:

`

$$\cos(\alpha) = x_0, \quad \sin(\alpha) = y_0$$

Như vậy với mọi góc α cho trước ta đều có thể biểu diễn thông qua các vector đơn vị trên đường tròn lượng giác

2.2.2. Phép xoay vector thật (Real Rotation)

Cho một vector ban đầu là $\vec{a} = (x, y)$ trên hệ tọa độ Oxy với bán kính là R và góc α. Khi quay vector $\vec{a}$ này một góc θ quanh gốc tọa độ, ta thu được vector mới là $\vec{b} = (x', y')$ có thể được biểu diễn như sau:

$$a = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} R.\cos(\alpha) \\ R.\sin(\alpha) \end{bmatrix} \quad b = \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} R.\cos(\alpha + \theta) \\ R.\sin(\alpha + \theta) \end{bmatrix}$$

Sử dụng các công thức lượng giác:

$$\cos(\alpha + \beta) = \cos\alpha \,.\, cos\beta - \sin\alpha \,.\, sin\beta$$

$$\sin(\alpha + \beta) = \sin\alpha \,.\, cos\beta + \cos\alpha \,.\, sin\beta$$

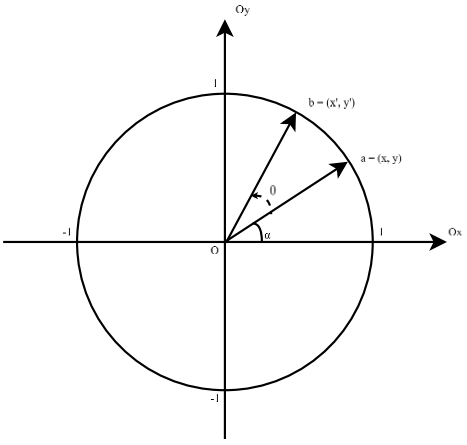
Ta có thể viết lại vector $\vec{b}$ ở dưới dạng như sau:

$$b = \begin{bmatrix} R.\cos(\alpha + \theta) \\ R.\sin(\alpha + \theta) \end{bmatrix} = \begin{bmatrix} R.\cos\alpha \,.\, cos\theta - \mathrm{R}.\sin\alpha \,.\, sin\theta \\ R.\sin\alpha \,.\, cos\theta + \mathrm{R}.\cos\alpha \,.\, sin\theta \end{bmatrix}$$

$$= \begin{bmatrix} x.cos\theta - \mathrm{y}.sin\theta \\ x.sin\theta + \mathrm{y}.cos\theta \end{bmatrix}$$

$$= \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix}$$

$$= \begin{bmatrix} x' \\ y' \end{bmatrix}$$



Hình 2.2. Phép xoay vector thật

`

Như vậy để có thể xác định được chính xác tọa độ mới của một vector sau khi quay vector này một góc θ quanh gốc tọa độ thì ta cần biết được các giá trị $cos\theta, sin\theta$ sau đó thực hiện phép nhân ma trận xoay trên với tọa độ của vector ban đầu. Điều này đòi hỏi tốn nhiều tài nguyên và phần cứng.

Chính vì những hạn chế trên, ta cần có một phương pháp tính toán thay thế nhằm giảm thiểu số lượng phép nhân. Thuật toán CORDIC được đề xuất có thể tính toán tọa độ mới sau khi xoay vector này mà không cần thực hiện bất kì phép nhân nào trên phần cứng thông qua việc xoay bằng vector giả.

2.2.3. Phép xoay vector giả (Pseudo Rotation)

Như chứng minh ở phép xoay vector thật, tọa độ vector mới được xác định theo công thức:

$$b = \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \begin{bmatrix} x' \\ y' \end{bmatrix}$$

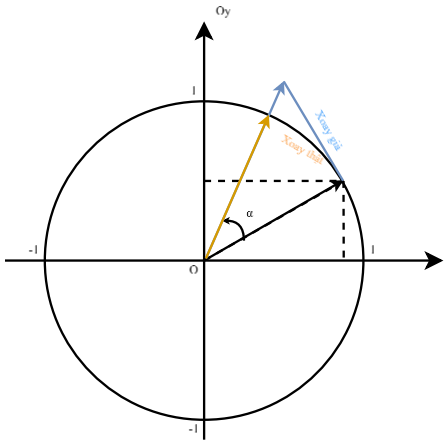
Sau khi đặt $cos\theta$ ra ngoài ma trận xoay (với $cos\theta$ khác 0), ta được biến đổi:

$$b = \begin{bmatrix} x' \\ y' \end{bmatrix} = cos\theta \begin{bmatrix} 1 & -tan\theta \\ tan\theta & 1 \end{bmatrix} \begin{bmatrix} x' \\ y' \end{bmatrix} = cos\theta . R(\theta) . \begin{bmatrix} x' \\ y' \end{bmatrix}$$

Khi nhân ma trận vừa biến đổi này (ma trận $R(\theta)$) cho tọa độ vector cũ (x', y') thì ta cần thực hiện các phép nhân giá trị $tan\theta$ với các giá trị x' và y'. Điều này vẫn đòi hỏi cần phải có sự tính toán của phép nhân hai số. Giả sử nếu $tan\theta = 2^{-n}$ (n > 0) thì ta chỉ cần dịch các giá trị x' và y' này sang phải n bit vì trong hệ cơ số 2 việc dịch phải n bit tương đương với việc nhân với 2^{-n}. Như vậy ta đã có thể tính được phép toán nhân ma trận với tọa độ vector ban đầu mà không thực hiện bất kì phép nhân nào.

Kết quả của tọa độ vector mới khi thực hiện phép nhân ma trận $R(\theta)$ cho tọa độ vector cũ (x', y') ta thu được vector mới có góc vector bằng với góc của vector khi xoay bằng phép xoay vector thật nhưng độ dài vector lại không bằng với độ dài của vector ban đầu, nguyên nhân là do kết quả sau khi tính chưa nhân với giá trị $cos\theta$ bên ngoài ma trận $R(\theta)$:

`



Hình 2.3. Phép xoay vector giả

Như vậy khi thực hiện phép xoay giả cho một vector, ta chỉ có thể xoay vector với các góc xoay là $\arctan(2^{-i})$, với i là số nguyên. Giá trị của vector thu được sau khi xoay phép xoay giả không còn được giữ nguyên độ dài ban đầu. Nếu muốn thu được kết quả chính xác như khi xoay thật thì phải nhân với giá trị của $\cos(\arctan(2^{-i}))$ với $\arctan(2^{-i})$ là giá trị góc xoay.

2.2.4. Ý tưởng thuật toán CORDIC tính sin và cos

Để tính được giá trị của sin và cos của góc z cho trước, ta cần xác định được tọa độ vector (x, y) của góc z này trên đường tròn lượng giác. Khi đó giá trị $cos(z) = x$ và $sin(z) = y$. Thuật toán CORDIC sử dụng ý tưởng tương tự như thuật toán tìm kiếm nhị phân góc z này trên miền góc của đường tròn lượng giác này trên phạm vi góc trên đường tròn bằng cách thực hiện chuỗi bước xoay giả từ vector gốc ban đầu với các góc xoay tại mỗi bước thứ i là $\arctan(2^{-i})$.

Qua nhiều bước xoay liên tiếp, hướng của vector dần tiệm cần và cuối cùng trúng với vector tạo bởi góc z cần xác định.

Khai triển tích các ma trận của phép xoay giả, ta có được công thức tính giá trị vector (x_{i+1}, y_{i+1}) sau khi xoay vector (x_i, y_i) bằng phép xoay giả với góc là $\theta_i = \arctan(2^{-i})$ ở mỗi lần lặp thứ i:

$$x_{i+1} = x_i - y_i . d_i . 2^{-i}$$

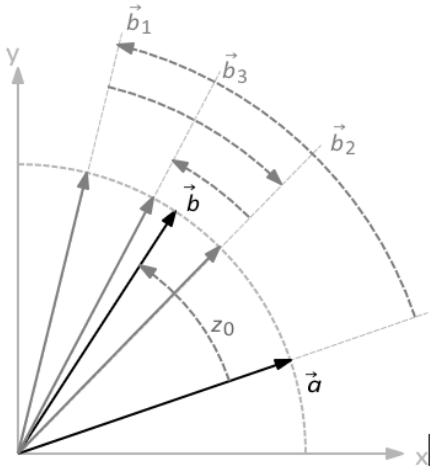
$$y_{i+1} = y_i + x_i . d_i . 2^{-i}$$

$$z_{i+1} = z_i - d_i . \theta_i$$

Với giá trị của d_i sẽ phụ thuộc vào giá trị của z_i ở mỗi lần lặp thứ i, cụ thể d_i = 1 nếu $z_i \geq 0$ và ngược lại d_i = -1 nếu $z_i < 0$, với i là số lượng các bước xoay xác định.

`

Sau khi hiện xong các vòng lặp, giá trị z là góc cần xoay lúc này bằng 0, ta nhân vector thực hiện chuỗi xoay giả này với hệ số tỉ lệ K để thu được vector quay thật.



Hình 2.4. Chuỗi xoay giả từ vector gốc

2.2.5. Hệ số tỷ lệ scale K

Như ý tưởng trên, để tránh phải sử dụng phép nhân và tính toán với giá trị cos của góc xoay tại bước i khi thiết kế mạch, các phép xoay vector toàn bộ đều là phép xoay giả. Mỗi lần thực hiện xoay giả, tọa độ vector mới không được nhân với giá trị cos tương ứng của góc xoay. Hệ quả là sau mỗi bước lặp, độ lớn của vector bị sai lệch; qua nhiều lần xoay liên tiếp, sai lệch này bị tích lũy, dẫn đến giá trị vector thu được ở đầu ra không còn chính xác

Tuy nhiên ta nhận thấy rằng thay vì mỗi bước ta nhân với giá trị cos của góc xoay, ta có thể tính trước tích các giá trị cos của các góc xoay, hay còn gọi là hệ số tỷ lệ cân bằng K. Hệ số này cho phép bù lại sai lệnh về độ lớn do các phép xoay giả gây ra, cụ thể, chỉ cần nhân kết quả thu được sau chuỗi xoay giả với hệ số K, ta sẽ thu được vector có độ dài chính xác tương đương với phép xoay hình học thực.

$$K = \prod_{i=0}^{n} \cos(\arctan(2^{-i})) \approx 0.60725$$

Từ chứng minh trên ta đã tính được hệ số tỷ lệ cân bằng là K = 0.60725. Sau đó ta nhân hệ số K này vào vector gốc ban đầu thực hiện xoay nhằm bù sai lệnh về độ lớn do các phép xoay giả gây ra. Giả sử ở đây ta chọn vector gốc ban đầu có tọa độ là (1,0). Sau khi nhân với K, ta thu được tọa độ vector ban đầu dùng để xoay cho giải thuật CORDIC là (K,0).

`

2.2.6. Bảng tra cứu góc arctan 2^{-i} LUT (Lookup Table)

Với mỗi bước xoay thứ i của thuật toán CORDIC, cần xác định được giá trị của các góc xoay tương ứng, cụ thể là của góc $\arctan(2^{-i})$. Do độ lớn góc xoay tại mỗi bước đã được xác định trước và không thay đổi (chỉ khác chiều xoay) là $\arctan(2^{-i})$, các giá trị góc $\arctan(2^{-i})$ được lưu trữ trong một bảng tra cứu (Lookup Table – LUT):

i	$\tan(\theta_i) = 2^{-i}$	$\arctan(2^{-i}) = \theta_i$ (radian)
0	2^{-0}	0.78539816
1	2^{-1}	0.46364761
2	2^{-2}	0.24497866
3	2^{-3}	0.12435500
4	2^{-4}	0.06241880
5	2^{-5}	0.03123983
6	2^{-6}	0.01562373
7	2^{-7}	0.00781234
8	2^{-8}	0.00390623
9	2^{-9}	0.00195312
10	2^{-10}	0.00097656
11	2^{-11}	0.000488
12	2^{-12}	0.000244
13	2^{-13}	0.000122

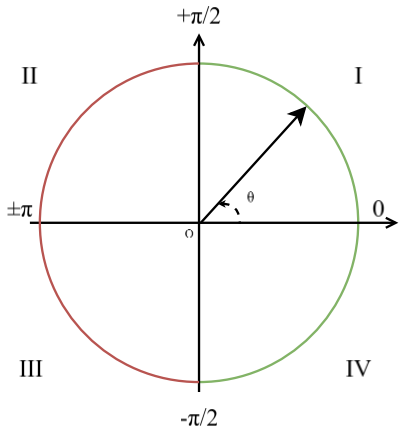
Bảng 2.1. Bảng LUT các giá trị góc $arctan(2^{-i})$

2.2.7. Giới hạn về phạm vi góc tính và giải pháp

Với mỗi bước xoay thứ i của thuật toán CORDIC, ta có thể xoay góc $\arctan(2^{-i})$ từ vị trí vector hiện tại. Nếu như số lần xoay vector càng nhiều, $i \to \infty$ tổng các góc có thể xoay là:

$$\sum_{i=0}^{\infty} \theta_i = \sum_{i=0}^{\infty} \arctan(2^{-i}) \approx 1.7433\ (rad) = 99{,}7°$$

Như vậy tổng các góc lớn nhất có thể xoay tới của phương pháp CORDIC chỉ giới hạn xấp xỉ khoảng 99,7°. Xét trên đường tròn lượng giác, chọn vector gốc là (1,0) ta có thể chọn góc phần tư thứ I ($0° \to 90°$) và góc phần tư thứ IV ($-90° \to 0°$) làm phạm vi có thể xoay được từ vector gốc này qua các bước xoay bằng thuật toán CORDIC:



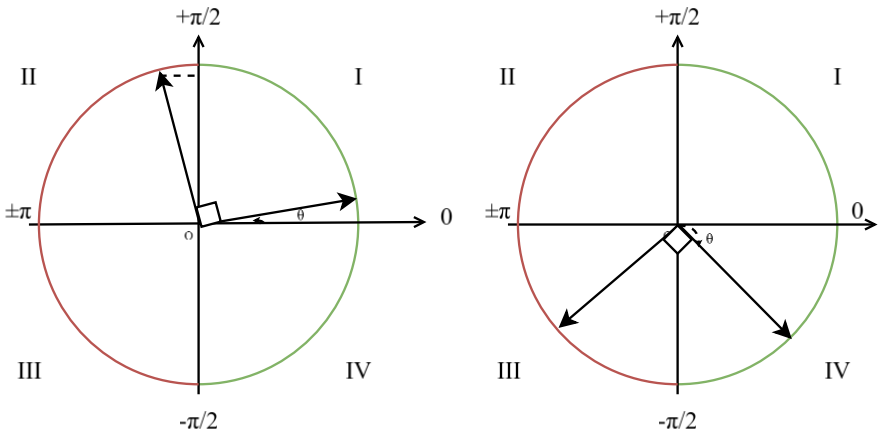
Hình 2.5. Phạm vi góc xoay của thuật toán CORDIC

Hiện tại góc phạm vi của góc z nhập vào cần tính giá trị $sin(z)$ và $cos(z)$ chỉ nằm ở phạm vi $-\frac{\pi}{2} < z < \frac{\pi}{2}$ (góc phần tư thứ I và thứ IV). Nếu như ta nhập các góc thuộc góc phần tư thứ II và thứ III thì ta không thể tính toán giá trị sin và cos của các góc này bằng cách xoay vector như thuật toán CORDIC được vì không thuộc tầm quay tới được.

Để có thể tính các giá trị sin và cos của các góc thuộc phần tư thứ II và thứ III ta sẽ xử lí để đưa các góc này vào phạm vi mà thuật toán CORDIC có thể xoay bằng cách sử dụng công thức hơn kém nhau $\frac{\pi}{2}$ sau đây:

$$\cos\left(\alpha + \frac{\pi}{2}\right) = -\sin\alpha$$

$$\sin\left(\alpha + \frac{\pi}{2}\right) = \cos\alpha$$



Hình 2.6. Biến đổi vector ở góc phần tư thứ II và thứ IV về vector ở góc phần tư thứ I và IV.

`

Ta thấy rằng nếu góc z nhập vào thuộc góc phần tư thứ II, ta sẽ trừ đi góc $\frac{\pi}{2}$ để thu được vector mới có góc là ∅ thuộc góc phần tư thứ I sau đó sử dụng thuật toán CORDIC để tính giá trị X = cos(∅) và Y = sin(∅), sử dụng công thức hơn kém nhau ở trên ta có thể tính được giá trị sin(z) và cos(z) ban đầu:

$$\cos(z) = Y \text{ và } \sin(z) = -X$$

Tương tự nếu góc nhập z nhập vào thuộc góc phần tư thứ III, ta sẽ cộng thêm góc $\frac{\pi}{2}$ để thu được vector mới có góc là ∅ thuộc góc phần tư thứ IV và giá trị sin(z) và cos(z) ban đầu sau khi tính bởi vector mới này là:

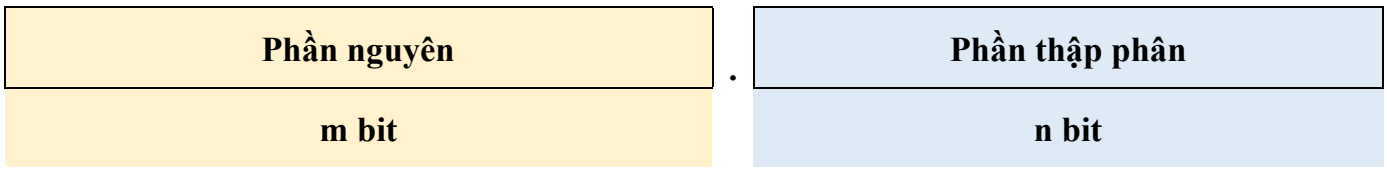
$$\cos(z) = -Y \text{ và } \sin(z) = X$$

Như vậy ta đã có thể tính được các giá trị của tất cả các góc trên đường tròn từ 0° đến 360° thông qua thuật toán CORDIC và các biến đổi mở rộng như trên.

2.3. Số Fixed Point

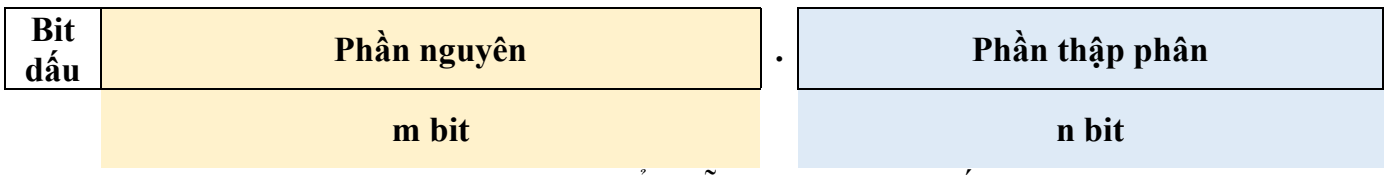
2.3.1. Biểu diễn số thực bằng Fixed Point

Để giảm chi phí, tiết kiệm phần cứng so với biểu diễn số dấu chấm động (floating point) khi tính toán với số thực, ta thường sử dụng một dạng số lưu trữ số fixed point có thể biểu diễn số thực bằng số nguyên, từ đó ta có thể tính toán số thực này dựa trên những khối tính toán với số nguyên. Biểu diễn fixed point sử dụng số bit cố định để biểu diễn phần nguyên và phần thập phân được ngăn cách ngầm bởi dấu chấm động:



Hình 2.7. Biểu diễn fixed point không dấu

Biểu diễn số không dấu như trên ký hiệu là UQm.n. Ngoài ra ta có thể biểu diễn số thực âm bằng cách biểu diễn số fixed point có dấu ở dạng bù 2:



Hình 2.8. Biểu diễn fixed point có dấu

Đối với biểu diễn số nguyên không dấu ta kí hiệu là Qm.n, với số lượng các bit kể cả bit dấu là m + n + 1 bit.

`

Giả sử ta có số fixed point ở dạng Q3.4 có giá trị là 1010.0000_2 thì ta có thể chuyển đổi về giá trị số hệ cơ số 10 bằng cách biểu diễn trực tiếp theo trọng số bit như sau:

Bit dấu	Phần nguyên				Phần thập phân			
1	0	1	0	.	1	1	1	0
-2^3	2^2	2^1	2^0		2^{-1}	2^{-2}	2^{-3}	2^{-4}

Giá trị khi biểu diễn ở hệ cơ số 10 của số Q3.4 là:

$$-2^3.1 + 2^1.1 + 2^{-1}.1 + 2^{-2}.1 + 2^{-3}.1 = -5.125$$

Ngoài cách chuyển theo như trên ta có thể xem số fixed point Q3.4 1010.0000_2 là một số nguyên có dấu 8 bit. Khi đó, giá trị số thực được xác định bằng cách chia số nguyên này cho 2^4 vì số bit biểu diễn phần thập phân dùng là 4 bit. Giá trị số thực được xác định:

$$\frac{-82}{2^4} = -5.125$$

Như vậy ta nhận thấy rằng bản chất của số fixed point định dạng Qm.n thực chất vẫn là số nguyên có dấu. Điểm khác biệt nằm ở cách diễn giải giá trị, ta ngầm hiểu là giá trị thực sẽ bằng giá trị nguyên này chia cho 2^n, với n là số lượng bit ta quy định là phần thập phân.

Giá trị khi ta chia cho 2^n với n là số bit biểu diễn phần thập phân được gọi là hệ số (caling factor) là giá trị dùng để quy đổi giữa số nguyên lưu trong phần cứng và giá trị thực mà nó biểu diễn. Như vậy với số fixed point định dạng Qm.n, ta có giá trị số thực mà nó biểu diễn là:

$$A \,.\, 2^{-n}$$

Trong đó:

A: là số nguyên có dấu được lưu trữ

2^{-n} : hệ số tỷ lệ (scaling factor) cũng chính là độ chia nhỏ nhất có thể biểu diễn chính xác được ở phần thập phân.

2.3.2. Phép toán số học với Fixed Point

Như đã trình bày, số fixed point bản chất được lưu trữ ở dạng số nguyên nên các phép toán cộng, trừ có thể thực hiện giống như số nguyên mà không ảnh hưởng đến biểu diễn giá thực. Xét hai số fixed point có cùng định dạng Qm.n, được biểu diễn dưới dạng:

X = $A \,.\, 2^{-n}$ và Y = $B \,.\, 2^{-n}$ với A và B là giá trị ở dạng số nguyên có dấu (m + n + 1) bit.

`

Thực hiện phép cộng hoặc trừ hai số fixed-point ta được:

$$X + Y = A \,.\, 2^{-n} + B \,.\, 2^{-n} = (A + B).\, 2^{-n}$$

$$X - Y = A \,.\, 2^{-n} - B \,.\, 2^{-n} = (A - B).\, 2^{-n}$$

Ta thấy rằng khi thực hiện phép cộng hoặc trừ thì kết quả (A+B) và (A-B) vẫn là một số nguyên và hệ số (scaling factor) vẫn là 2^n , giá trị thu được của số fixed point mới sau khi tính vẫn giữ nguyên định dạng Qm.n. Như vậy việc thực hiện phép cộng, trừ trên số fixed point giống như cách thực hiện như trên số nguyên.

2.3.3. Xác định tầm biểu diễn của số Fixed Point 16 bit vào mạch thiết kế

Để cho quá trình tính toán không bị tràn và vẫn đảm bảo được độ chính xác cao ta cần xác định tầm biểu diễn của số fixed-point. Trong mạch thiết số, ta sẽ sử dụng fixed-point có độ dài là 16 bit với định dạng Q3.13 để biểu diễn các giá trị đầu vào, giá trị tính toán trung gian và kết quả đầu ra.

Khoảng giá trị của các số thực tính toán trong mạch nằm ở phạm vi góc đầu vào từ $-\pi$ đến$n\ \pi$ (-3.14; 3.14) và giá trị tính sin và cos nằm từ (-1; 1) nên ta sẽ sử dụng định dạng Q3.13 là hợp lý. Định dạng Q3.13 hoàn toàn có thể biểu diễn các giá trị thực trong khoảng (-4, 4) với độ chia nhỏ nhất là 2^{13}.

CHƯƠNG 3. THIẾT KẾ MẠCH TÍNH SIN VÀ COS

3.1. Mã giả (Pseudo-code)

Từ chứng minh ở chương 2, ta có được công thức tính giá trị vector (x_{i+1}, y_{i+1}) sau khi xoay vector (x_i, y_i) bằng phép xoay giả với góc là $\theta_i = \arctan(2^{-i})$ ở mỗi lần lặp thứ i:

$$x_{i+1} = x_i - y_i . d_i . 2^{-i}$$

$$y_{i+1} = y_i + x_i . d_i . 2^{-i}$$

$$z_{i+1} = z_i - d_i . \theta_i$$

Với giá trị của d_i sẽ phụ thuộc vào giá trị của z_i ở mỗi lần lặp thứ i, cụ thể d_i = 1 nếu $z_i \geq 0$ và ngược lại d_i = -1 nếu $z_i < 0$, với i là số lượng các bước xoay xác định cho trước trong giải thuật này được chọn là 14.

`

Để tính giá trị sin(z) và cos(z) với giá trị đầu vào góc z, ta tiến hành xoay vector gốc ban đầu theo thuật toán CORDIC. Vector gốc được lựa chọn là (1,0). Quá trình tính toán sẽ thực hiện 14 lần xoay giả theo các công thức đã trình bày như trên.

Tuy nhiên, do mỗi bước xoay trong CORDIC là xoay giả nên độ dài của vector không được bảo toàn nên để thu được vector chính xác ta phải nhân kết quả cuối cùng với hệ số tỷ lệ K. Nhằm tránh việc sử dụng bất kỳ phép nhân nào trong mạch tính toán ta phải xử lý hệ số tỷ lệ K này trước tính toán bằng cách tính trước phép nhân hệ số K này với vector gốc là (1, 0).

Như vậy ta cần khởi tạo giá trị x ban đầu là hệ số tỷ lệ K và $y = 0$. Sau khi thực hiện xong số vòng lặp, kết quả x và y thu được chính là giá trị của hàm sin (z) và cos (z) với z là góc nhập ban đầu.

Ta xây dựng mã giả cho giải thuật tính sin và cos bằng CORDIC như sau:

```
z = Inport Z
x = const K = 0.607252935…
y = 0
i = 0
do{
        t1 = x >> i;
        t2 = y >> i;
        if (zMSB == 0) {        // z >= 0
                x = x – t2;
                y = y + t1;
                z = z – Arctan[i];
        }
        else {
                x = x + t2;
                y = y - t1;
                z = z + Arctan[i];
```

`

```
        }
        i = i + 1;
} while (i < 14){
Output Cos = x;
Output Sin = y;
```

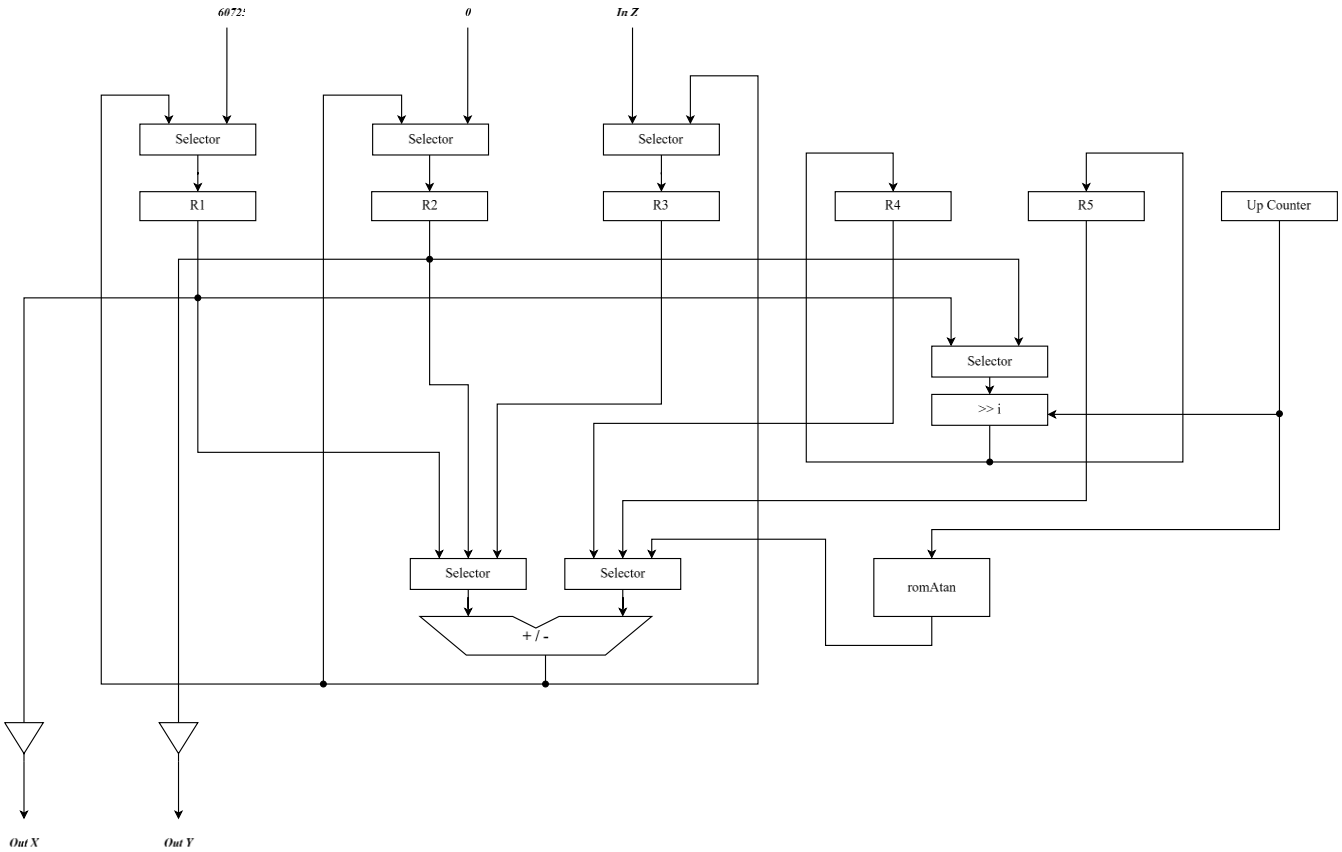
Hình 3.1. Mã giả giải thuật tính sin và cos dựa trên thuật toán CORDIC

3.2. Xây dựng Datapath đơn giản

Từ mã giả được xây dựng, ta nhận thấy rằng Datapath cần một số khối chức năng và thành phần như sau:

- Bộ tính toán cộng, trừ 16 bit
- Thanh ghi lưu giá trị các biến 16 bit
- Bộ dịch n bit, với n được đưa vào có thể lên đến 14 bit.
- Bộ nhớ lưu trước giá trị $\arctan(2^{-i})$ gồm 14 giá trị.
- Khối lựa chọn (MUX) các giá trị
- Bộ đếm lên của giá trị biến i lên đến 14.

Ta tiến hành xây dựng một Datapath đơn giản:



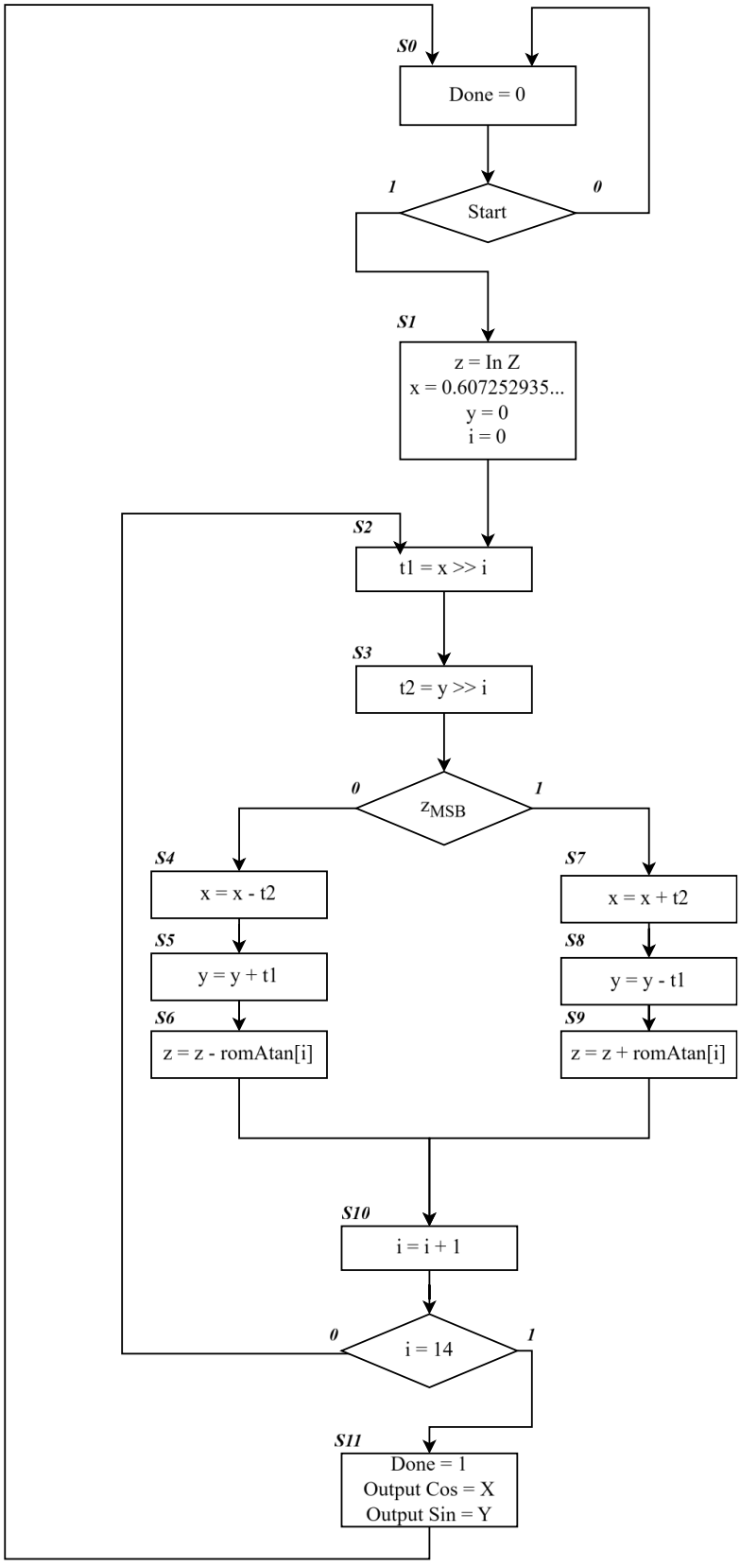
Bảng 3.1. Xây dựng Datapath đơn giản

Giả sử ở đây, phần cứng của ta khá giới hạn, ta sử dụng 1 khối AU, 1 bộ dịch n bit và 1 bộ nhớ ROM để tính toán cho toàn mạch. Ta không thể thực hiện tính toán cộng trừ cùng lúc các giá trị x, y, z mà phải sử dụng chung khối chức năng này này để thực hiện lần lượt các phép tính ở mỗi trạng thái riêng biệt.

3.3. Xây dựng sơ đồ ASM

Từ thuật toán CORDIC với mã giả và những thành phần đã xây dựng ở Datapath, ta thành lập được sơ đồ giải thuật ASM:

`



Hình 3.2. Sơ đồ ASM

Cụ thể, sơ đồ ASM bao gồm 11 trạng thái như sau:

- Trạng thái s_0: Xác nhận tín hiệu Start, nếu Start được bật thì mới chuyển sang trạng thái nhập dữ liệu:

`

- Trạng thái s_1: Nhập dữ liệu góc z cần tính sin(z), cos(z) và khởi tạo các biến và bộ đếm. Các biến này được lưu trong các thanh ghi vì vậy khi đang ở trạng thái s_1, dữ liệu trong các thanh ghi lưu các biến này vẫn chưa được nạp vào, phải chờ tới xung Clock tiếp theo thì dữ liệu mới được nạp vào các thanh ghi ở trạng thái tiếp theo.
- Trạng thái s_2 đến trạng thái s_9: Thực hiện các công việc tính toán ở mỗi vòng lặp i, ở mỗi vòng lặp, thuật toán sẽ chỉ thực hiện theo sơ đồ chuyển trạng thái $s_2 \rightarrow s_3 \rightarrow s_4 \rightarrow s_5 \rightarrow s_6$ nếu góc z_i tại vòng lặp này dương hoặc bằng 0 ($z_{MSB} = 0$), hoặc chuyển theo sơ đồ trạng thái $s_2 \rightarrow s_3 \rightarrow s_7 \rightarrow s_8 \rightarrow s_9$ nếu góc z_i mang giá trị âm. Như vậy việc tính toán ở mỗi vòng lặp i sẽ tốn 5 chu kì.
- Trạng thái s_{10}: Tăng giá trị biến lặp i. Sau khi đã thực hiện tính toán xong các phép tính ở vòng lặp thứ i, ta điều khiển cho bộ đếm tăng thêm 1. Tuy nhiên, trong trạng thái s_{10}, giá trị của bộ đếm vẫn chưa được nạp mới mà phải chờ đến xung Clock tiếp theo bộ đếm mới tăng lên. Mặt khác, theo mã giả ta thực hiện tính toán trước rồi mới kiểm tra giá trị i sau nên mạch dùng để kiểm tra điều kiện dừng ta sẽ thiết kế bằng cách kiểm tra giá trị i với 13. Như vậy với mỗi vòng lặp i, ta cần 6 chu kỳ để thực hiện bao gồm 5 chu kì tính toán và 1 chu kì dùng tăng biến lặp.
- Trạng thái s_{11}: Xuất giá trị kết quả. Bật tín hiệu Done = 1.

Như vậy, với mỗi giá trị góc z nhập vào, tổng số chu kì cần thiết để mạch thu được kết quả là 87 chu kỳ bao gồm:

- 1 chu kỳ xác nhận tín hiệu Start
- 1 chu kỳ nạp dữ liệu đầu vào và khởi tạo
- 84 chu kỳ cho 14 lần lặp i, với mỗi vòng lặp tốn 6 chu kỳ
- 1 chu kỳ xuất kết quả.

3.4. Tối ưu phần cứng với Register Sharing

Từ sơ đồ ASM đã xây dựng, nhận thấy cần sử dụng phải sử dụng 5 thanh ghi dùng để lưu trữ 5 biến gồm x, y, z, t_1, t_2. Riêng đối với biến vòng lặp i, giá trị này được lưu trữ và cập nhật thông qua bộ đếm, do đó không cần sử dụng thanh ghi riêng. Ta lập bảng thời gian sống của các biến để xem xét khả năng rút gọn và chia sẻ thanh ghi:

`

	s_2	s_3	s_4/s_7	s_5/s_8	s_6/s_9	s_{10}	s_{11}
x	x	x	x	x	x	x	x
z	x	x	x	x	x	x	
y	x	x	x	x	x	x	x
t_1		x	x	x			
t_2			x				
Living variable	3	4	5	4	3	3	2

Bảng 3.2. Bảng thời gian sống các biến

Thời gian sống của các biến x, y, z là toàn bộ chương trình bởi vì mỗi bước lặp ta cần phải sử dụng lại giá trị cũ của nó để tính toán ra giá trị x, y, z mới và kết quả Output lại là giá trị của biến x, y.

Từ bảng thời gian sống trên, ta nhận thấy tại trạng thái s_5/s_8, cả 5 biến đều được sử dụng nên ta không thể rút gọn thêm thanh ghi nào.

Ta đặt nhãn cho các biến:

R1 = x

R2 = y

R3 = z

R4 = t_1

R5 = t_2

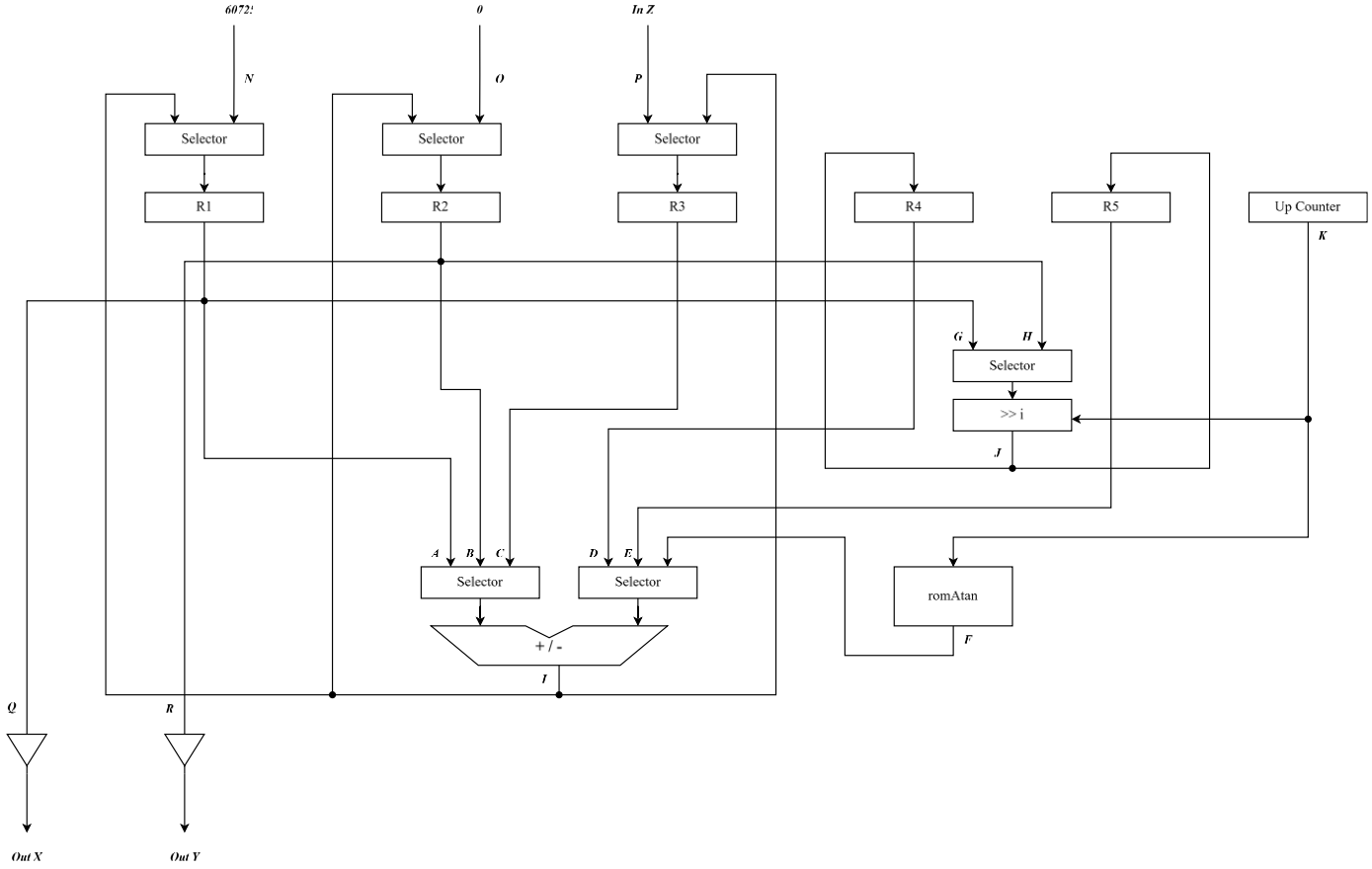
3.5. Tối ưu phần cứng với Bus Sharing

Từ sơ đồ Datapath đã xây dựng, ta thấy rằng để lựa chọn giá trị ghi vào của các thanh ghi hoặc lựa chọn giá trị của các toán hạng đưa vào của các khối chức năng như AU, bộ dịch, … thì với mỗi ngõ vào này ta phải lựa chọn dữ liệu đưa vào tương ứng với từng trạng thái, dẫn đến việc mỗi thanh ghi phải có đường bus dữ liệu riêng, làm tăng số lượng dây kết nối và tốn nhiều MUX để lựa chọn dữ liệu.

Ta thấy rằng tại mỗi trạng thái của sơ đồ ASM, chỉ có một số thanh ghi nhất định cần xuất dữ liệu ra Datapath để thực hiện phép toán, còn các thanh ghi khác không tham gia vào quá trình xử lý tại thời điểm đó. Để giảm số lượng bus kết nối và MUX lựa chọn nguồn dữ liệu, ta có thể sử dụng các đường bus chung để sử dụng tại mỗi trạng thái, cho phép nhiều thanh ghi có nhiều nguồn dữ liệu cùng dùng chung một bus, với điều kiện tại mỗi thời điểm chỉ có một nguồn duy

nhất được phép sử dụng đường bus này. Do đó, các thanh ghi R1, R2, R3, R4, R5 có thể chia sẻ chung một số bus dữ liệu thay vì mỗi thanh ghi dùng một bus riêng.

Từ sơ đồ kết nối các khối của Datapath, ta đánh tên những đường bus kết nối với nhau theo hai nhóm bao gồm: nhóm bus Output (là các bus đi về thanh ghi dùng để ghi dữ liệu) và nhóm bus Input (là các bus từ các thanh ghi truyền vào các khối chức năng để xử lý hoặc từ khối chức năng vào khối chức năng khác).



Hình 3.3. Đánh nhãn cho các đường kết nối dữ liệu

Sau khi đã đặt tên cho các đường kết nối, ta chia thành 2 nhóm:

- Bus A, B, C, D, E, F, G, H, K, Q, R là nhóm bus Input
- Bus I, J, N, O, P là nhóm bus Output

Lập bảng sử dụng các đường kết nối (connectivity usage) cho nhóm bus Input:

	s_1	s_2	s_3	s_4/s_7	s_5/s_8	s_6/s_9	s_{10}	s_{11}
A				x				
B					x			

`

C						x		
D					x			
E				x				
F						x		
G		x						
H			x					
K		x	x			x		
Q								x
R								x
Living bus		2	2	2	2	3	3	2

Bảng 3.3. Bảng sử dụng các kết nối Input tại mỗi trạng thái (Connectivity usage)

Từ bảng Connectivity usage của nhóm bus Input trên, sử dụng thuật toán Left-Edge ta gom được các nhóm bus Input có thể chia sẻ với nhau như sau:

Bus 1 = {A, B, C, G, H, Q}

Bus 2 = {D, E, R, F}

Bus 3 = {K}

Tương tự đối với nhóm bus Output, ta tiến hành lập bảng Connectivity usage:

	s_1	s_2	s_3	s_4/s_7	s_5/s_8	s_6/s_9	s_{10}	s_{11}
I				x	x	x		
J		x	x					
N	x							
O	x							
P	x							
Living bus	3	1	1	1	1	1	0	0

Bảng 3.4. Bảng sử dụng các kết nối Output tại mỗi trạng thái (Connectivity usage)

`

Từ bảng Connectivity usage của nhóm bus Output trên, sử dụng thuật toán Left-Edge ta gom được các nhóm bus Output có thể chia sẻ với nhau như sau:

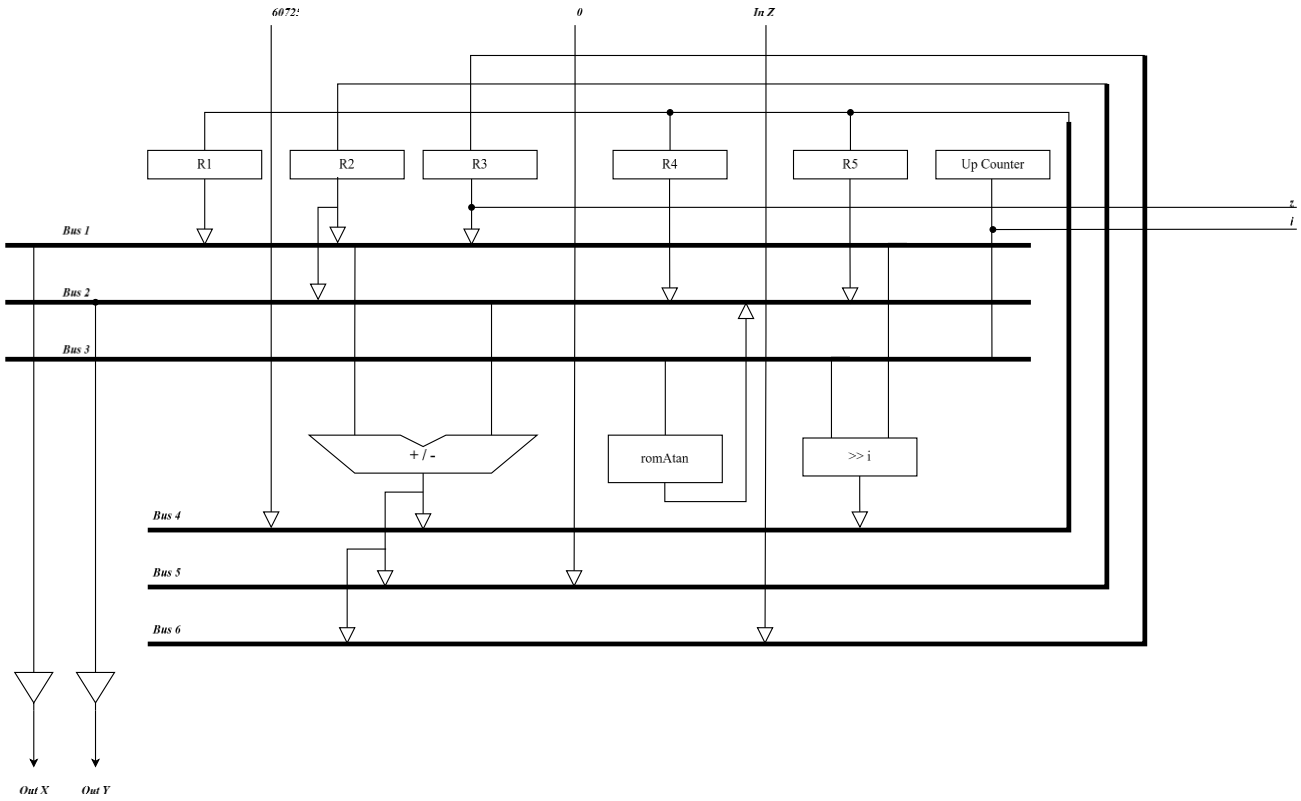
Bus 4 = {N, I, J}

Bus 5 = {O}

Bus 6 = {P}

Sau khi đã gom được các đường bus lại với nhau, ta tiến hành thêm các cổng Tri-State cho các đường liệu khi sử dụng các bus:

- Bus 1 là đường bus dữ liệu của R1, R2, R3 nên cần 3 Tri-State để tránh xung đột và tín hiệu điều khiển các Tri-State này lần lượt là TriR1, TriR2, TriR3.
- Bus 2 là đường bus dữ liệu của R2 để khi xuất kết quả, R4, R5 và khối romArctan nên cần 4 Tri-State để tránh xung đột và tín hiệu điều khiển các Tri-State này lần lượt là TriR2_Out, TriR4,TriR5 và TriRom.
- Bus 3 là đường bus duy nhất của bộ đếm nên không cần Tri-State.
- Bus 4 là đường bus của các dữ liệu khởi tạo hằng số K, khối AU và khối dịch phải nên cần 3 Tri-State để tránh xung đột và tín hiệu điều khiển các Tri-State này lần lượt là TriConstK, TriAU và TriShift. Bus 4 được nối trực tiếp với ngõ nạp của các thanh ghi R1, R4, R5 để khi có trạng thái tương ứng, giá trị của Bus 4 sẽ được sử dụng để ghi vào các thanh ghi này.
- Bus 5 là đường bus của các dữ liệu giá trị 0 và kết quả của AU nên cần 2 Tri-State được điều khiển bởi TriZero, TriAU. Bus 5 được nối trực tiếp với ngõ nạp của thanh ghi R2 để khi có trạng thái tương ứng, giá trị của Bus 5 sẽ được sử dụng để ghi vào các thanh ghi R2.
- Bus 6 là đường bus của các dữ liệu giá trị góc z cần tính nhập vào và kết quả của AU nên cần 2 Tri-State được điều khiển bởi TriZ, TriAU. Bus 6 được nối trực tiếp với ngõ nạp của thanh ghi R3 để khi có trạng thái tương ứng, giá trị của Bus 6 sẽ được sử dụng để ghi vào các thanh ghi R3.



Hình 3.4. Datapath sau khi đã tối ưu bằng kĩ thuật Bus Sharing

Như vậy sau khi tối ưu phần cứng bằng kĩ thuật Bus Sharing, ta đã tiết kiệm được 4 MUX 2-1 data 16 bit và 2 MUX 4-1 data 16 bit bằng cách thay bằng 14 khối Tri-State cho Datapath.

3.6. Thiết kế các thành phần của Datapath

3.6.1. Thanh ghi (Register) 16 bit

Thanh ghi 16-bit được thiết kế từ 16 Flip-Flop D mắc song song, mỗi Flip-Flop lưu trữ một bit dữ liệu. Tất cả các Flip-Flop được điều khiển bởi cùng một xung clock, đảm bảo quá trình lưu trữ dữ liệu diễn ra đồng bộ.

Thanh ghi sử dụng tín hiệu điều khiển Load_Enable để cho phép có được nạp dữ liệu vào thanh ở xung clock tiếp theo hay không:

- Khi Load_Enable = 0: Các Flip Flop giữ nguyên giá trị hiện tại.
- Khi Load_Enable = 1: Tại cạnh tích cực của xung clock, dữ liệu đầu vào sẽ được nạp vào các Flip Flop.

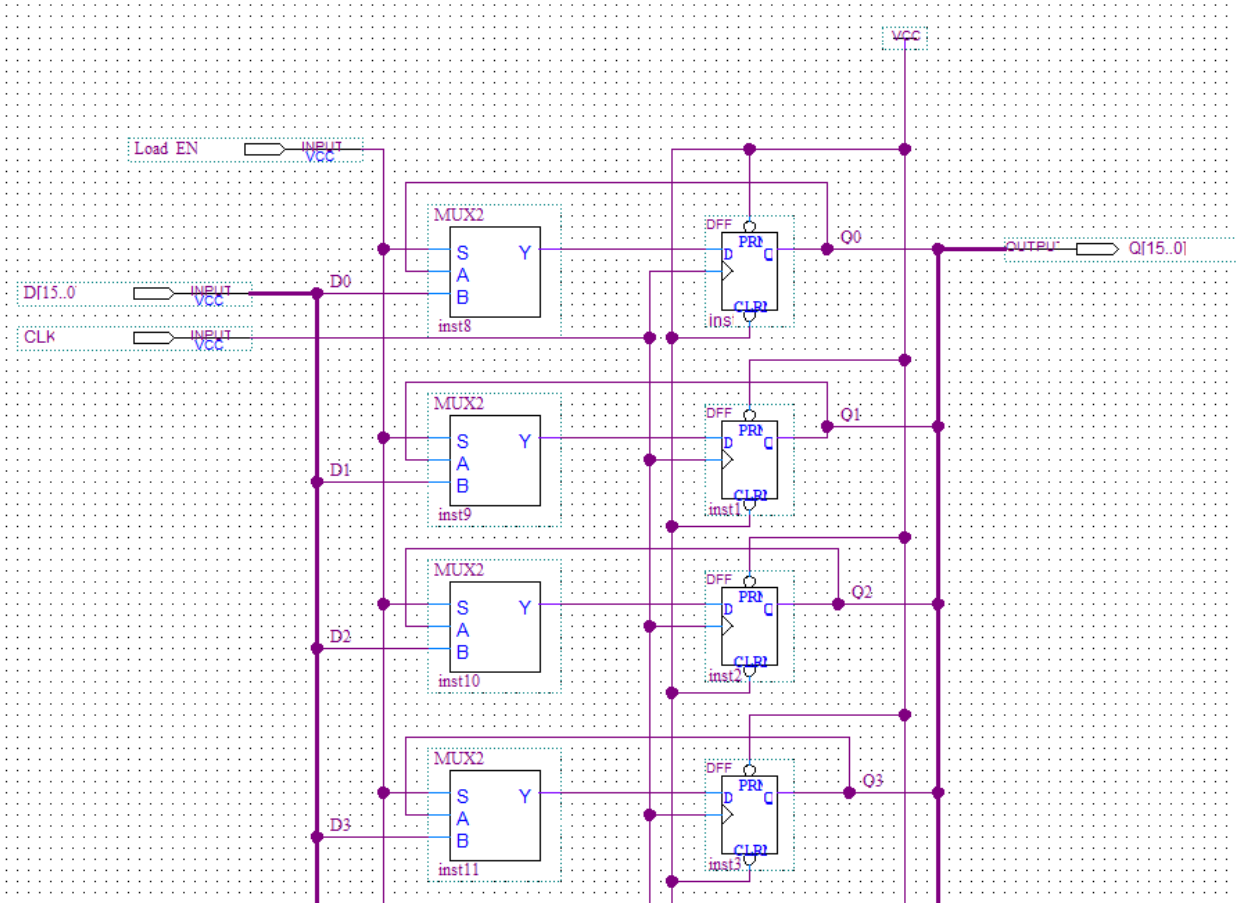
Việc lựa chọn giá trị ngõ vào cho các Flip Flop được các MUX 2:1 thực hiện.

Bảng sự thật của thanh ghi:

`

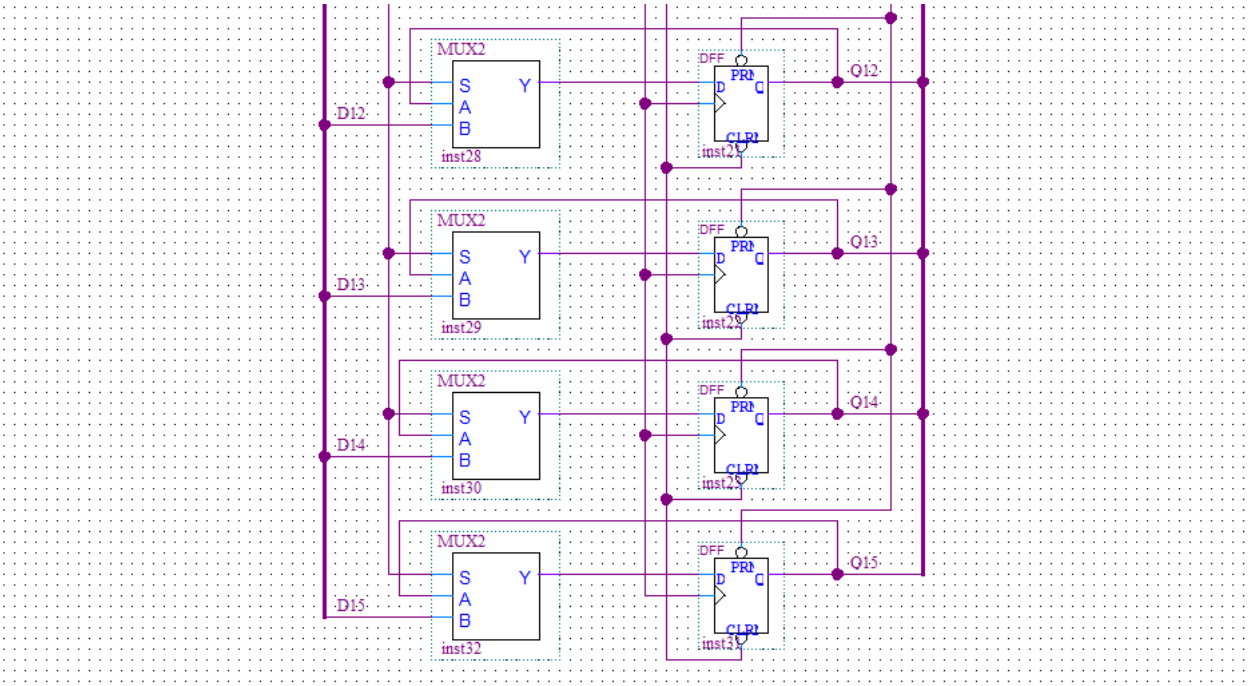
Load	Chức năng
0	Pass
1	Load

Bảng 3.5. Bảng chức năng của thanh ghi

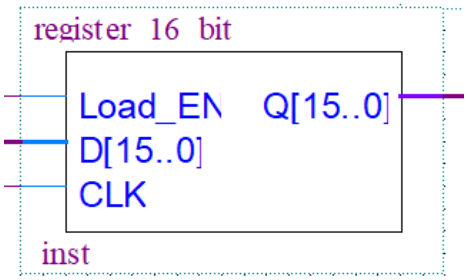


…

`



Bảng 3.6. Vẽ mạch thanh ghi 16 bit



Bảng 3.7. Đóng gói thanh ghi 16 bit

3.6.2. Khối AU

Khối tính toán trong mạch thực hiện hai phép toán là cộng và trừ.

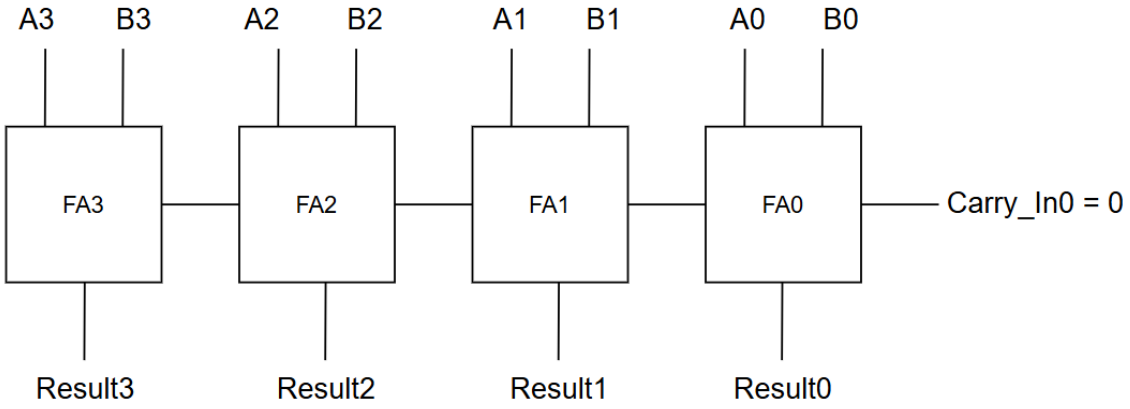
Bảng sự thật của khối AU:

Sel	Chức năng
0	A + B
1	A - B

Bảng 3.8. Bảng chức năng của khối AU

Để thực hiện phép toán cộng 2 số nhị phân, ta sử dụng các mạch Full Adder (FA) mắc nối tiếp nhau theo cấu trúc Ripple Carry Adder như hình:

`

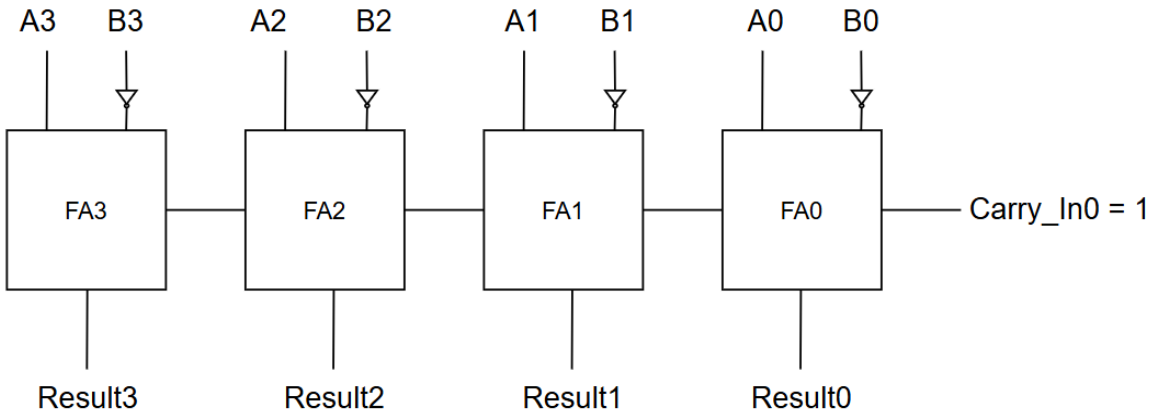


Bảng 3.9. Mạch cộng sử dụng Full Adder

Đối với phép toán trừ 2 số nhị phân, ta áp dụng phương pháp bù hai (two's complement):

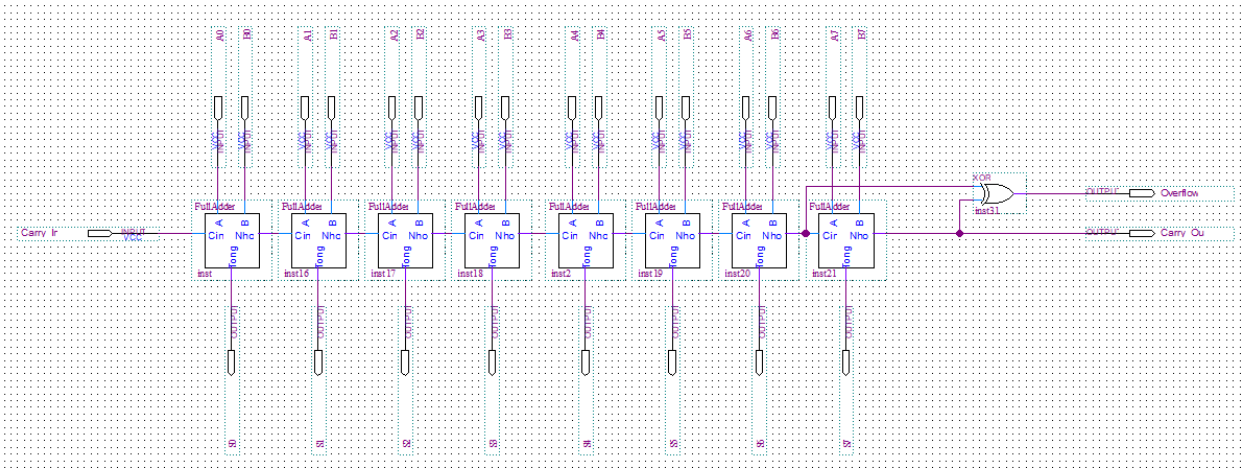
$$A - B = A + (\text{bù 2 của } B) = A + (\text{bù 1 của } B) + 1 = A + \bar{B} + 1$$

Vì vậy ta chỉ cần dùng mạch cộng đã có ở phép toán cộng và chuyển B thành B' sau đó cộng 1 bằng cách sử dụng Carry_In thấp nhất đặt lên giá trị là 1.

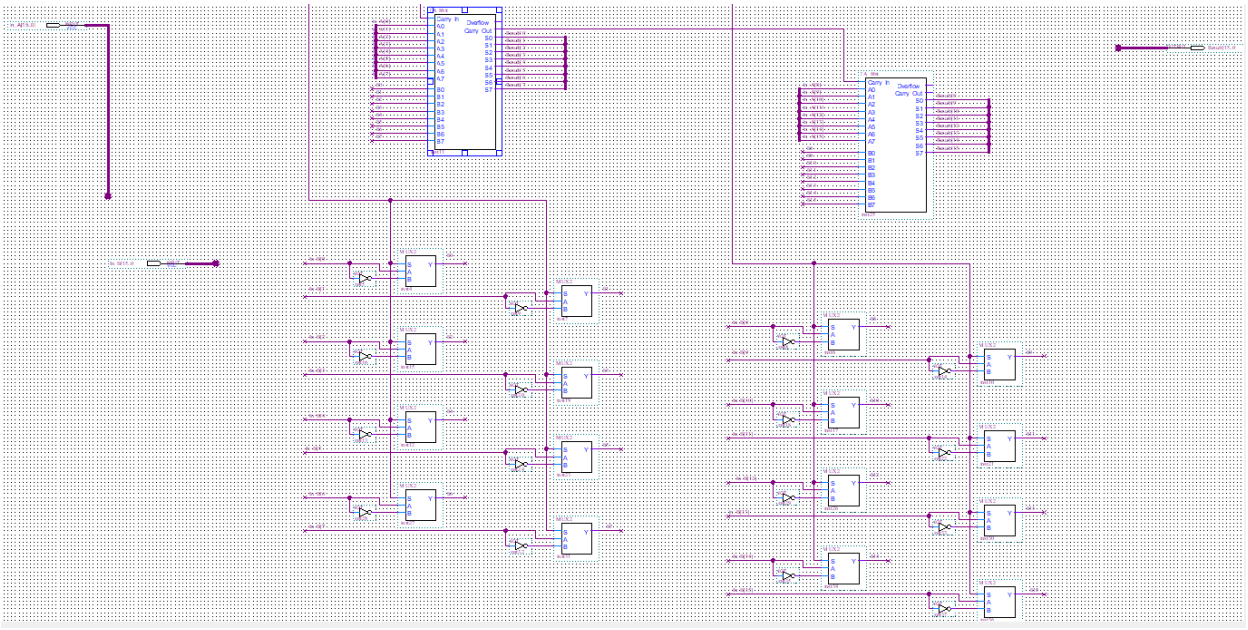


Bảng 3.10. Mạch trừ sử dụng Full Adder

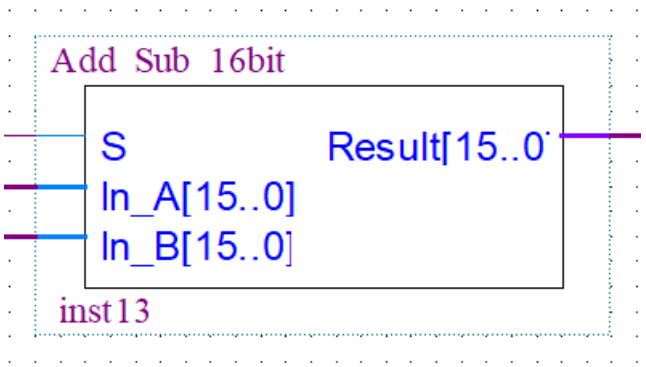
Ta nhận thấy rằng tín hiệu lựa chọn phép toán Sel cũng chính bằng giá trị của Carry_In.



Bảng 3.11. Vẽ mạch bộ cộng 8 bit



Bảng 3.12. Vẽ mạch cộng trừ 16 bit được ghép từ 2 bộ cộng trừ 8 bit



Hình 3.5Đóng gói mạch cộng trừ 16 bit

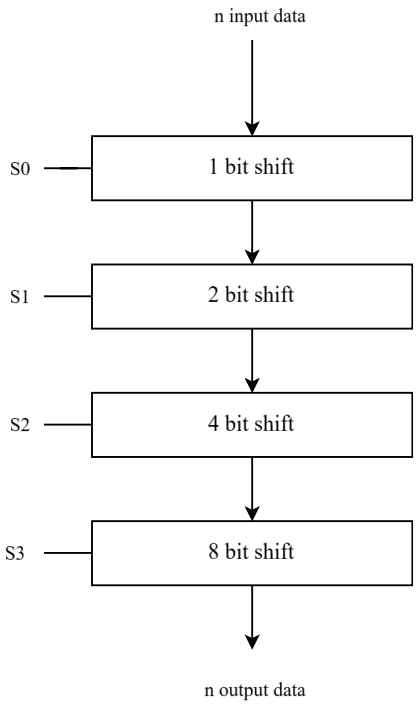
`

3.6.3. Bộ dịch phải n bit

Thông thường để thiết kế bộ dịch bit, ta thường nghĩ đến sử dụng các MUX để lựa chọn giá trị cuối cùng cho từng bit của dữ liệu cần dịch. Tuy nhiên đối với bộ dịch n bit ở đây, n có thể lên đến 14 bit, nếu ta sử dụng MUX 16-1 để lựa chọn giá trị mỗi bit của dữ liệu đưa vào thì số lượng MUX 16-1 phải dùng là 16x14 = 224 MUX 16-1. Điều này rất tốn chi phí.

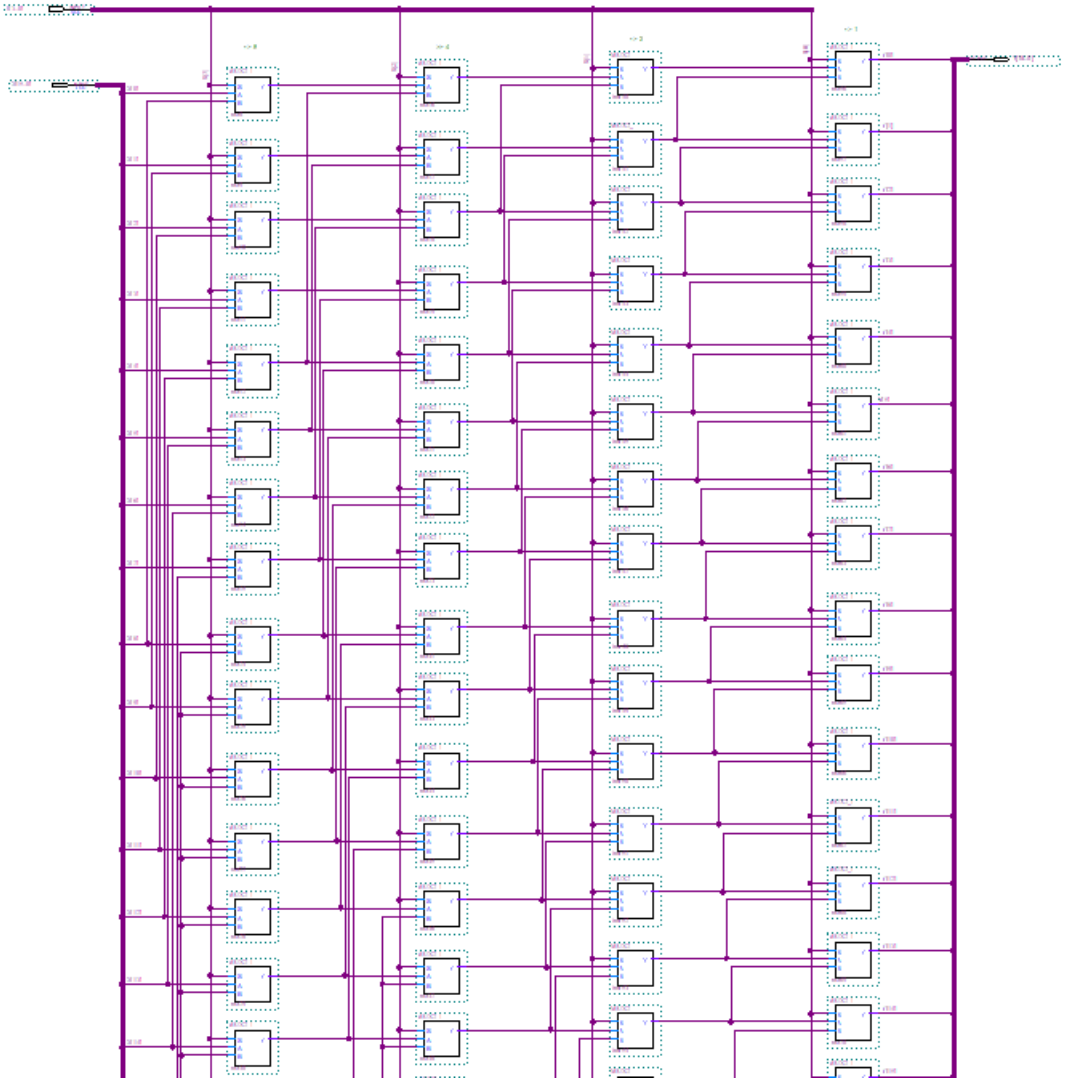
Ta nhận thấy rằng, để dịch n bit ta có thể chia nhỏ việc dịch. Ví dụ để dịch 13 bit, ta sẽ dịch 8 bit, lấy kết quả này dịch tiếp 4 bit, sau đó lại lấy kết quả này dịch tiếp 0 bit, rồi dịch 1 bit. Như vậy ta đã dịch được 8 + 4 + 0 + 1 = 13 bit. Việc dịch các bước theo cơ số lũy thừa 2 như trên rất giống với việc tính giá trị số nhị phân sang số thập phân của giá trị bit dịch (13 tương ứng với 1101).

Từ đó, ta có thể chia khối dịch thành các tầng nhỏ, mỗi tầng sẽ chỉ dịch được 1 số cố định là lũy thừa cơ số 2. Khi dữ liệu đi qua mỗi tầng, mỗi tầng có dịch hay không sẽ phụ thuộc vào mỗi bit tương ứng của giá trị cần dịch với tầng đó.

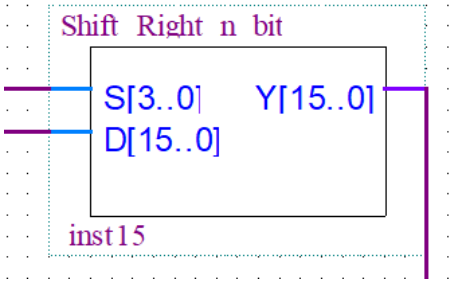


Hình 3.6. Ý tưởng thiết kế bộ dịch nhị phân

`



Hình 3.7. Vẽ mạch cho bộ dịch phải n bit



Hình 3.8. Đóng gói bộ dịch phải n bit

3.6.4. ROM (Read Only Memory)

ROM là bộ nhớ chỉ đọc dùng để lưu các giá trị đã được thiết lập sẵn. Đối với mạch ASIC này ta cần sử dụng ROM làm bảng LUT (Lookup Table) dùng để lưu giá trị cố định của các góc arctan (2^{-i}) giúp truy xuất trực tiếp được góc xoay trong mỗi bước xoay giả. Vì số bước lặp i

`

là 14, ta cần xây dựng ROM với kích thước là 14x16 bit lưu các giá trị góc xoay dưới dạng 16 bit fixed-point Q2.13 như sau:

i	$\tan(\theta_i) = 2^{-i}$	$\arctan(2^{-i}) = \theta_i$ (radian)	Giá trị lưu trong ROM[i] dạng Q2.13
0	2^{-0}	0.78539816	0001 1001 0010 0010
1	2^{-1}	0.46364761	0000 1110 1101 0110
2	2^{-2}	0.24497866	0000 0111 1101 0111
3	2^{-3}	0.12435500	0000 0011 1111 1011
4	2^{-4}	0.06241880	0000 0010 0000 0000
5	2^{-5}	0.03123983	0000 0001 0000 0000
6	2^{-6}	0.01562373	0000 0000 1000 0000
7	2^{-7}	0.00781234	0000 0000 0100 0000
8	2^{-8}	0.00390623	0000 0000 0010 0000
9	2^{-9}	0.00195312	0000 0000 0001 0000
10	2^{-10}	0.00097656	0000 0000 0000 1000
11	2^{-11}	0.000488	0000 0000 0000 0100
12	2^{-12}	0.000244	0000 0000 0000 0010
13	2^{-13}	0.000122	0000 0000 0000 0001

Bảng 3.13. Bảng lưu giá trị các góc xoay giả $arctan(2^{-i})$

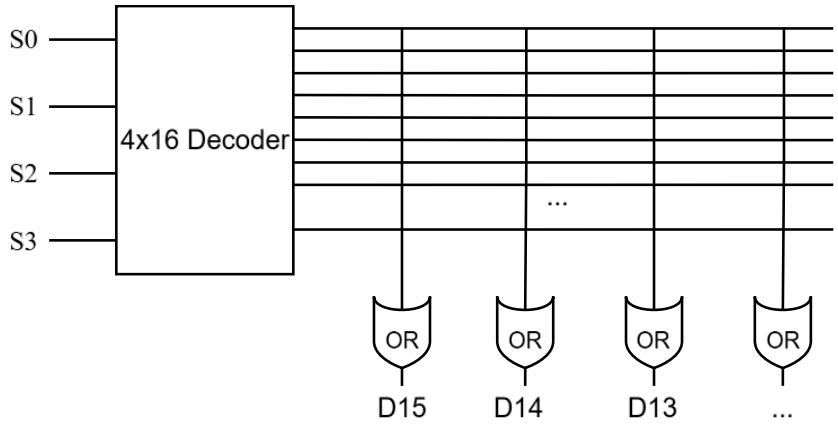
Khi thực hiện phép xoay ta có thể xoay vector hiện tại theo ngược chiều kim đồng hồ (tức góc xoay θ_i là dương), hoặc xoay theo thuật chiều kim đồng hồ (tức góc xoay θ_i là âm) tùy theo giá trị z_i nên ta phải lưu trong ROM cả hai giá trị của $\arctan(2^{-i}) = \pm\theta_i$ Tuy nhiên vì:

$$\tan(\pm\theta_i) = \pm\tan(\theta_i)$$

Ta chỉ cần lưu trong ROM danh sách giá trị của các xoay là góc dương, sau đó thực hiện cộng hoặc trừ phù hợp.

Một trong những phương pháp thiết kế ROM là sử dụng Decoder và logic SOP các ngõ ra của Decoder:

`



Hình 3.9. Ý tưởng thiết kế ROM từ decoder và các cổng logic OR

Decoder nhận đầu vào là địa chỉ cần lấy giá trị và giải mã ra một dòng chọn duy nhất. Mỗi bit dữ liệu đầu ra được biểu diễn dưới dạng hàm SOP của các tín hiệu Decoder.

Trong đồ án này, để thuận tiện cho việc thiết kế, thay vì thiết kế ROM bằng cách vẽ mạch, chúng em sẽ sử dụng Verilog để khai báo cho khối ROM thông qua đoạn code dưới đây:

```
module rom16_16_verilog (
    input  wire [3:0]  addr,
    output reg  [15:0] data
);

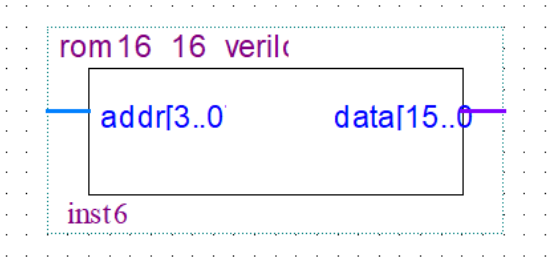

always @(*) begin
    case (addr)
        4'b0000: data = 16'b0001_1001_0010_0010;
        4'b0001: data = 16'b0000_1110_1101_0110;
        4'b0010: data = 16'b0000_0111_1101_0111;
        4'b0011: data = 16'b0000_0011_1111_1011;
        4'b0100: data = 16'b0000_0010_0000_0000;
        4'b0101: data = 16'b0000_0001_0000_0000;
        4'b0110: data = 16'b0000_0000_1000_0000;
        4'b0111: data = 16'b0000_0000_0100_0000;
```

`

```
        4'b1000: data = 16'b0000_0000_0010_0000;
        4'b1001: data = 16'b0000_0000_0001_0000;
        4'b1010: data = 16'b0000_0000_0000_1000;
        4'b1011: data = 16'b0000_0000_0000_0100;
        4'b1100: data = 16'b0000_0000_0000_0010;
        4'b1101: data = 16'b0000_0000_0000_0001;
        4'b1110: data = 16'b0000_0000_0000_0001;
        4'b1111: data = 16'b0000_0000_0000_0001;
        default: data = 16'b0000_0000_0000_0000;
    endcase
end


endmodule
```

Hình 3.10. Mã verilog thiết kế khối ROM



Hình 3.11. Đóng gói khối ROM

3.6.5. Bộ đếm

Với giá trị i trong mạch có thể lên đến 14, ta cần thiết kế bộ đếm 4 bit đếm lên. Mỗi lần có tín hiệu đếm và xung Clock thì giá trị hiện tại của bộ đếm sẽ tăng thêm 1 đơn vị. Ta chỉ cần sử dụng các Half Adder để thực hiện phép cộng 1. Giá trị hiện tại của bộ đếm sẽ được lưu trong các Flip Flop.

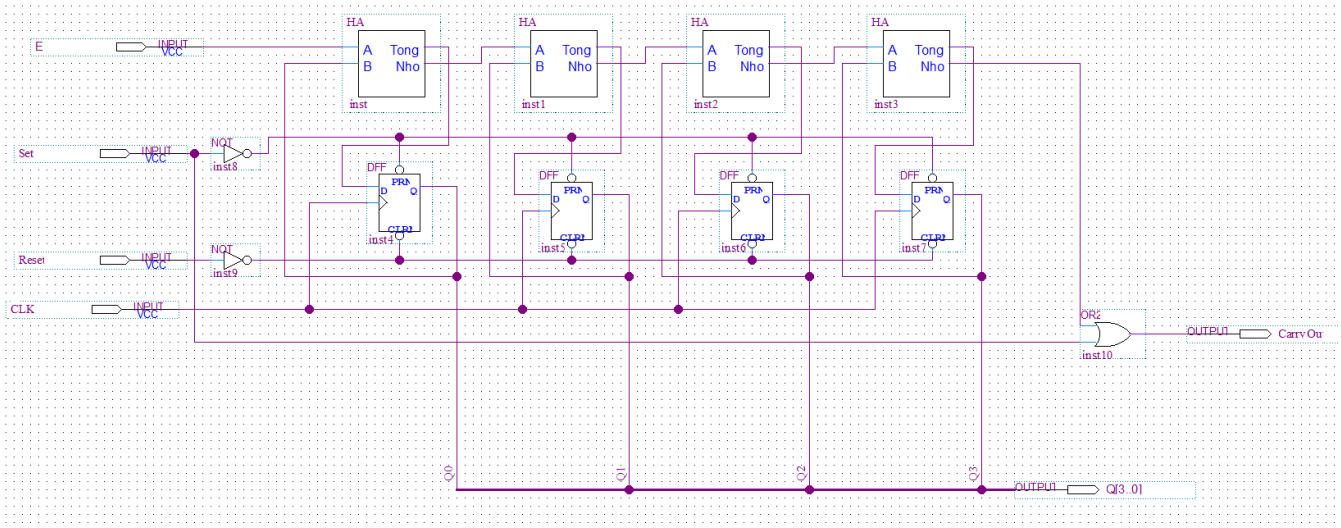
Ngoài ra, bộ đếm cần có chức năng Reset giá trị về 0 và Set để đặt giá trị lên 15.

Bảng sự thật của bộ đếm lên 4 bit:

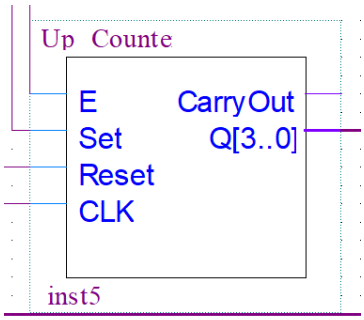
`

Reset	Set	E	Chức năng
1	0	X	Reset
0	1	X	Set
0	0	0	Pass
0	0	1	Tăng lên 1

Bảng 3.14. Bảng chức năng của bộ đếm



Hình 3.12. Thiết kế mạch cho bộ đếm lên 4 bit

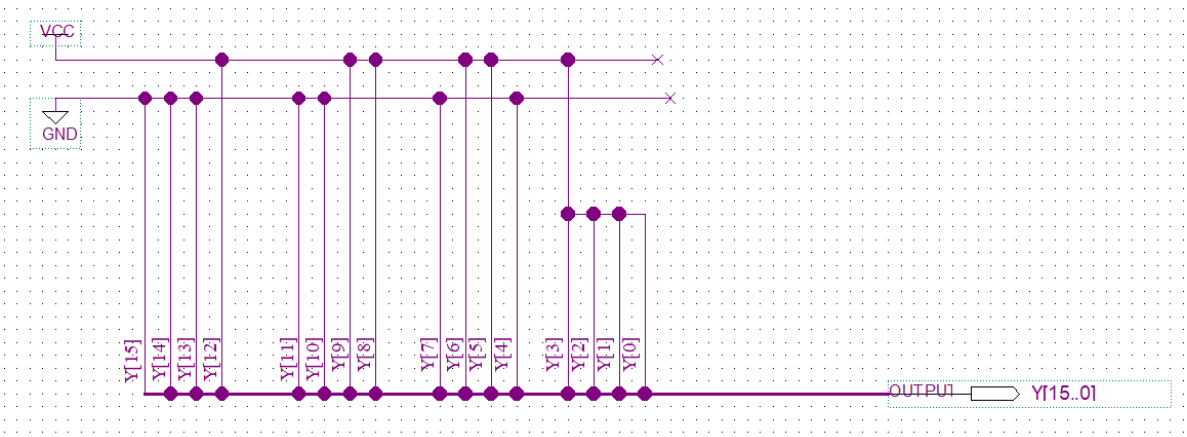


Hình 3.13. Đóng gói mạch cho bộ đếm lên 4 bit

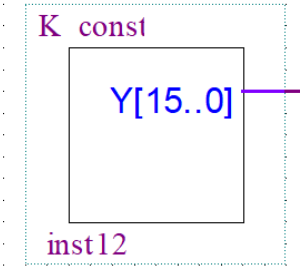
3.6.6. Các giá trị hằng số

3.6.6.1. Hằng số tỷ lệ K

Hằng số tỷ lệ K có giá trị là 0.60725 được biểu diễn sang số Fixed Point dạng Q2.13 là 0001001101101111.

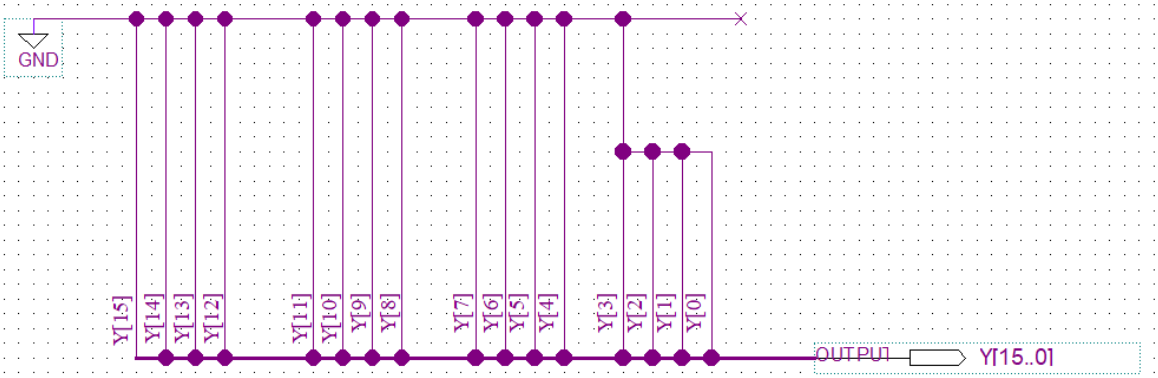


Hình 3.14. Vẽ mạch cho giá trị hằng K

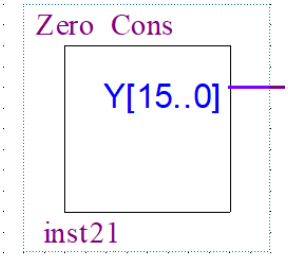


Hình 3.15. Đóng gói khối giá trị hằng K

3.6.6.2. Hằng số 0



Hình 3.16. Vẽ mạch cho giá trị hằng 0

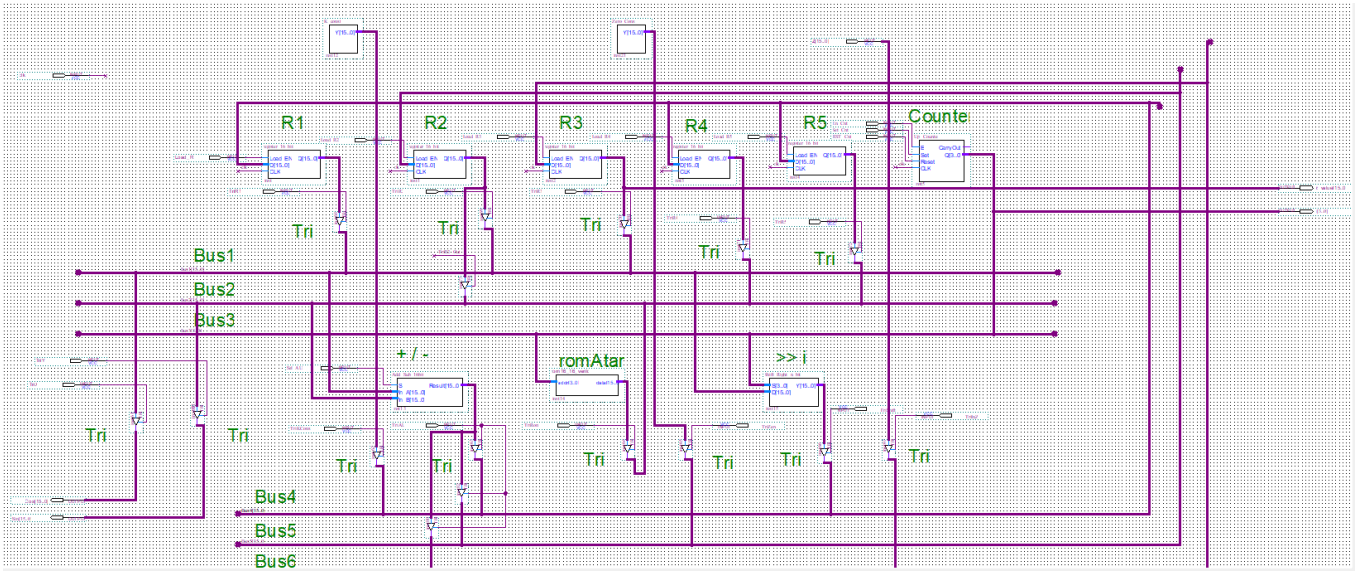


Hình 3.17. Đóng gói khối giá trị hằng 0

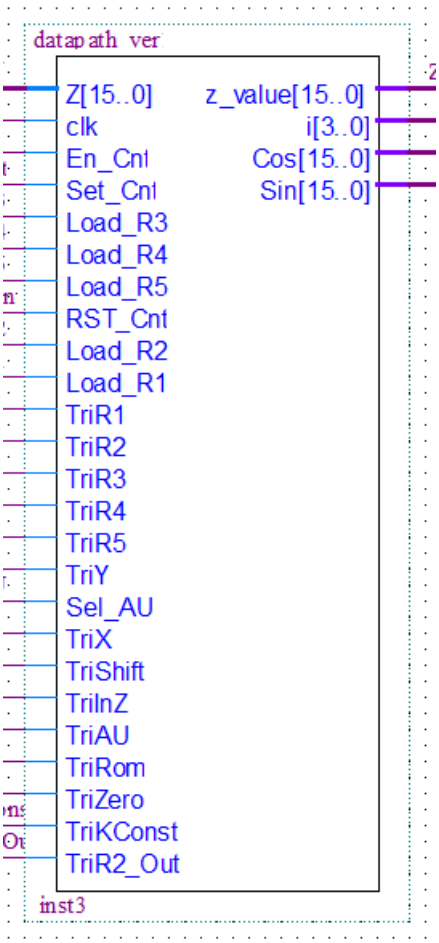
`

3.6.7. Hoàn thiện khối Datapath

Sau khi đã thiết kế và đóng gói các thành phần của Datapath trên mô phỏng Quartus, ta kết nối lại các thành phần này lại với nhau theo như sơ đồ:



Hình 3.18. Vẽ mạch Datapath hoàn thiện trên mô phỏng Quartus



Hình 3.19. Đóng gói Datapath

`

3.7. Thiết kế khối Controller

Sau khi đã xây dựng được khối Datapath, ta cần khối điều khiển (Controller) nhằm điều khiển trình tự hoạt động theo đúng sơ đồ ASM đã xây dựng và các tín hiệu điều khiển phát sinh của các thành phần trong Datapath.

3.7.1. Thiết kế máy trạng thái FSM

Từ sơ đồ ASM, ta xây dựng được bảng chuyển trạng thái:

Trạng thái hiện tại					**Trạng thái kế tiếp**					
Tên	Q_3	Q_2	Q_1	Q_0	Điều kiện	Tên	Q_3^+	Q_2^+	Q_1^+	Q_0^+
s_0	0	0	0	0	Start = 0	s_0	0	0	0	0
					Start = 1	s_1	0	0	0	1
s_1	0	0	0	1		s_2	0	0	1	0
s_2	0	0	1	0		s_3	0	0	1	1
s_3	0	0	1	1	$Z_{MSB} = 0$	s_4	0	1	0	0
					$Z_{MSB} = 1$	s_7	0	1	1	1
s_4	0	1	0	0		s_5	0	1	0	1
s_5	0	1	0	1		s_6	0	1	1	0
s_6	0	1	1	0		s_{10}	1	0	1	0
s_7	0	1	1	1		s_8	1	0	0	0
s_8	1	0	0	0		s_9	1	0	0	1
s_9	1	0	0	1		s_{10}	1	0	1	0
s_{10}	1	0	1	0	i = 16	s_{11}	1	0	1	1
					i ≠ 16	s_2	0	0	1	0
s_{11}	1	0	1	1		s_0	0	0	0	0

Bảng 3.15. Bảng chuyển trạng thái của FSM

Khi phân tích các trạng thái mã hóa trên, ta có thể giảm số lượng các cổng logic ở các trạng thái $s_4, s_5, s_6, s_7, s_8, s_9, s_{10}, s_{11}$ bằng cách mã hóa lần lượt là $Q_2Q'_1Q'_0$, $Q_2Q'_1Q_0$, $Q_2Q_1Q'_0$,

`

$Q_2Q_1Q_0$, $Q_3Q'_1Q'_0$, $Q_3Q'_1Q_0$, $Q_3Q_1Q'_0$, $Q_3Q_1Q_0$.

Từ bảng chuyển trạng thái trên ta thu được các phương trình trạng thái tiếp theo như sau:

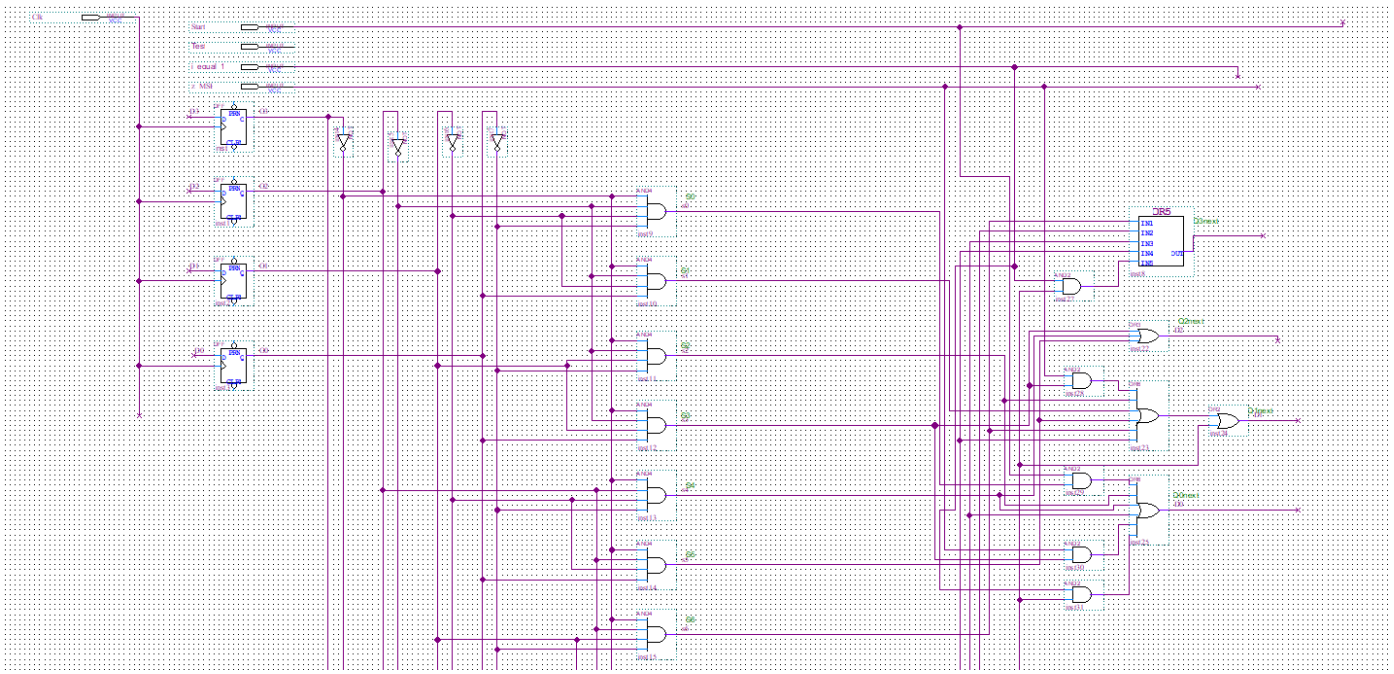
$$D_3 = Q_3^+ = s_6 + s_7 + s_8 + s_9 + s_{10}.(i = 16)$$
$$= Q_2Q_1Q'_0 + Q_2Q_1Q_0 + Q_3Q'_1Q'_0 + Q_3Q'_1Q_0 + Q_3Q_1Q'_0.(i = 16)$$

$$D_2 = Q_2^+ = s_3 + s_4 + s_5 = Q'_3Q'_2Q_1Q_0 + Q_2Q'_1Q'_0 + Q_2Q'_1Q_0$$

$$D_1 = Q_1^+ = s_1 + s_2 + s_3.(\text{zMSB} = 1) + s_5 + s_6 + s_9 + s_{10}$$
$$= Q'_3Q'_2Q'_1Q_0 + Q'_3Q'_2Q_1Q'_0 + Q'_3Q'_2Q_1Q_0.(\text{zMSB} = 1) + Q_2Q'_1Q_0$$
$$+ Q_2Q_1Q'_0 + Q_3Q'_1Q_0 + Q_3Q_1Q'_0$$

$$D_0 = Q_0^+ = s_0.(Start = 1) + s_2 + s_3.(\text{zMSB} = 1) + s_4 + s_8 + s_{10}.(i = 16)$$
$$= Q'_3Q'_2Q'_1Q'_0(Start = 1) + Q'_3Q'_2Q_1Q'_0 + Q'_3Q'_2Q_1Q_0.(\text{zMSB} = 1)$$
$$+ Q_2Q'_1Q'_0 + Q_3Q'_1Q'_0 + Q_3Q_1Q'_0.(i = 16)$$

.



Hình 3.20. Thiết kế mạch chuyển trạng thái

3.7.2. Thiết kế các tín hiệu điều khiển

Từ sơ đồ ASM, ta xác định các hành động của Datapath ứng với mỗi trạng thái:

`

Trạng thái hiện tại	Điều kiện	Hành động Datapath
s_0	Start = 0	
	Start = 1	
s_1		$z = In\ Z$ $x = 0.6072529$ $y = 0$ $i = 0$
s_2		$t_1 = x \gg i$
s_3	$z_{MSB} = 0$	$t_2 = y \gg i$
	$z_{MSB} = 1$	$t_2 = y \gg i$
s_4		$x = x - t_2$
s_5		$y = y + t_1$
s_6		$z = z - romArctan[i]$
s_7		$x = x + t_2$
s_8		$y = y - t_1$
s_9		$z = z + romArctan[i]$
s_{10}	i = 14	$i = i + 1$
	i ≠ 14	$i = i + 1$
s_{11}		$Output\ Cos = X$ $Output\ Sin = Y$

Bảng 3.16. Bảng hành động của Datapath

Lập bảng giá trị các tín hiệu điều khiển cấc thành phần của Datapath:

	Ngõ ra	s_0	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}	s_{11}
R1	LoadR1	x	1	0	0	1	0	0	1	0	0	0	0
	TriR1	x	0	1	0	1	0	0	1	0	0	0	1
R2	LoadR2	x	1	0	0	0	1	0	0	1	0	0	0
	TriR2	x	0	0	1	0	1	0	0	1	0	0	0
R3	LoadR3	x	1	0	0	0	0	1	0	0	1	0	0

`

	TriR3	x	0	0	0	0	0	1	0	0	1	0	0
R4	LoadR4	x	x	1	0	0	0	x	0	0	x	x	x
	TriR4	x	0	0	0	0	1	0	0	1	0	0	0
R5	LoadR5	x	x	0	1	0	0	x	0	0	x	x	x
	TriR5	x	0	0	0	1	0	0	1	0	0	0	0
AU	Sel_AU	x	x	x	x	1	0	1	0	1	0	x	x
	TriAU	x	0	0	0	1	1	1	1	1	1	0	0
ROM	TriRom	x	0	0	0	0	0	1	0	0	1	0	0
Shift Right	TriShift	x	0	1	1	0	0	0	0	0	0	0	0
Up Counter	En_Cnt	x	0	0	0	0	0	0	0	0	0	1	0
	Set_Cnt	x	0	0	0	0	0	0	0	0	0	0	0
	RST_Cnt	x	1	0	0	0	0	0	0	0	0	0	0
	TriR2_Out	0	0	0	0	0	0	0	0	0	0	0	1
	TriInZ	0	1	0	0	0	0	0	0	0	0	0	0
	TriZero	x	1	0	0	0	0	0	0	0	0	0	0
	TriKconst	x	1	0	0	0	0	0	0	0	0	0	0
	TriX	0	0	0	0	0	0	0	0	0	0	0	1
	TriY	0	0	0	0	0	0	0	0	0	0	0	1

Bảng 3.17. Bảng sự thật các tín hiệu ngõ ra điều khiển của khối Controller

Phương trình của các ngõ ra của khối Controller:

LoadR1 = s_1+ s_4 + s_7

TriR1 = $s_2 + s_4 + s_7 + s_{11}$

LoadR2 = $s_1 + s_5 + s_8$

TriR2 = $s_1 + s_3 + s_5 + s_8$

LoadR3 = $s_1 + s_6 + s_9$

TriRom = $s_6 + s_9$

TriShift = $s_2 + s_3$

En_Cnt = s_{10}

Set_Cnt = 0

RST_Cnt = s_1

`

TriR3 = $s_6 + s_9$

LoadR4 = s_2

TriR4 = $s_5 + s_8$

LoadR5 = s_3

TriR5 = $s_4 + s_7$

Sel_AU = $s_4 + s_6 + s_8$

TriAU = $s_4 + s_5 + s_6 + s_7 + s_8 + s_9$

TriR2_Out = s_{11}

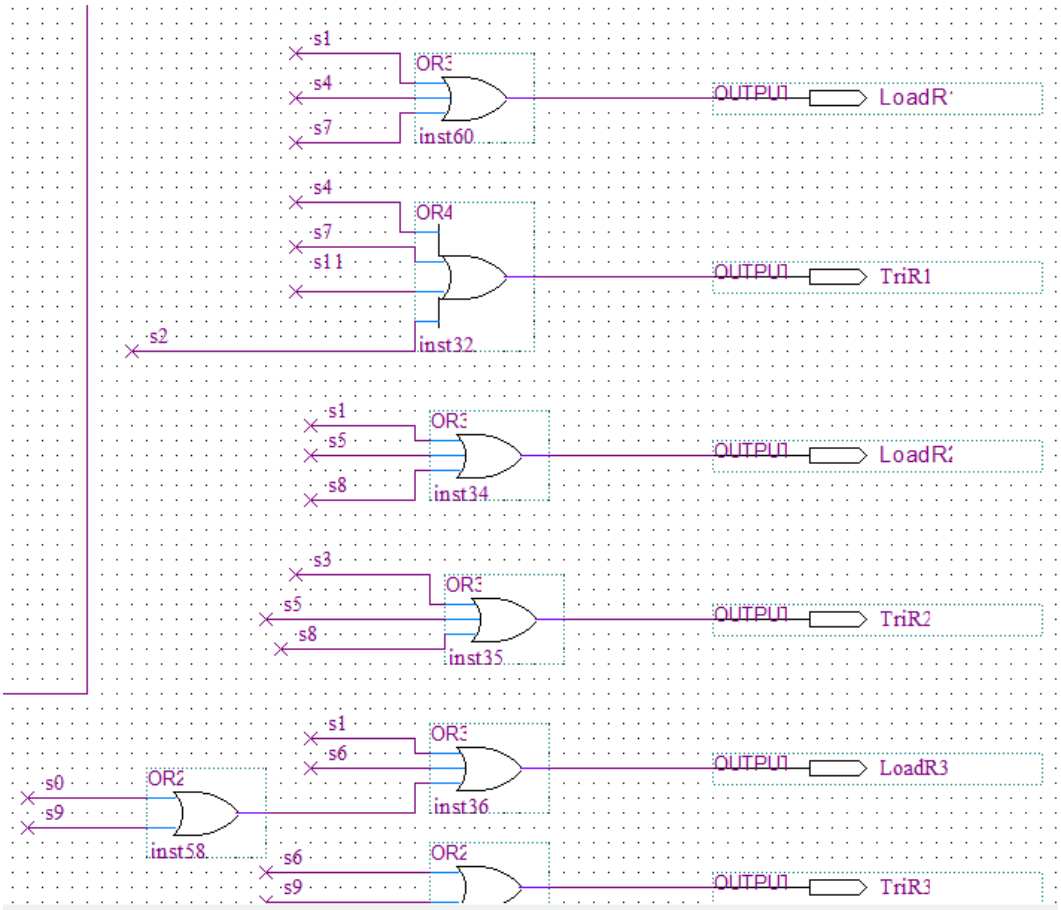
TriInZ = s_1

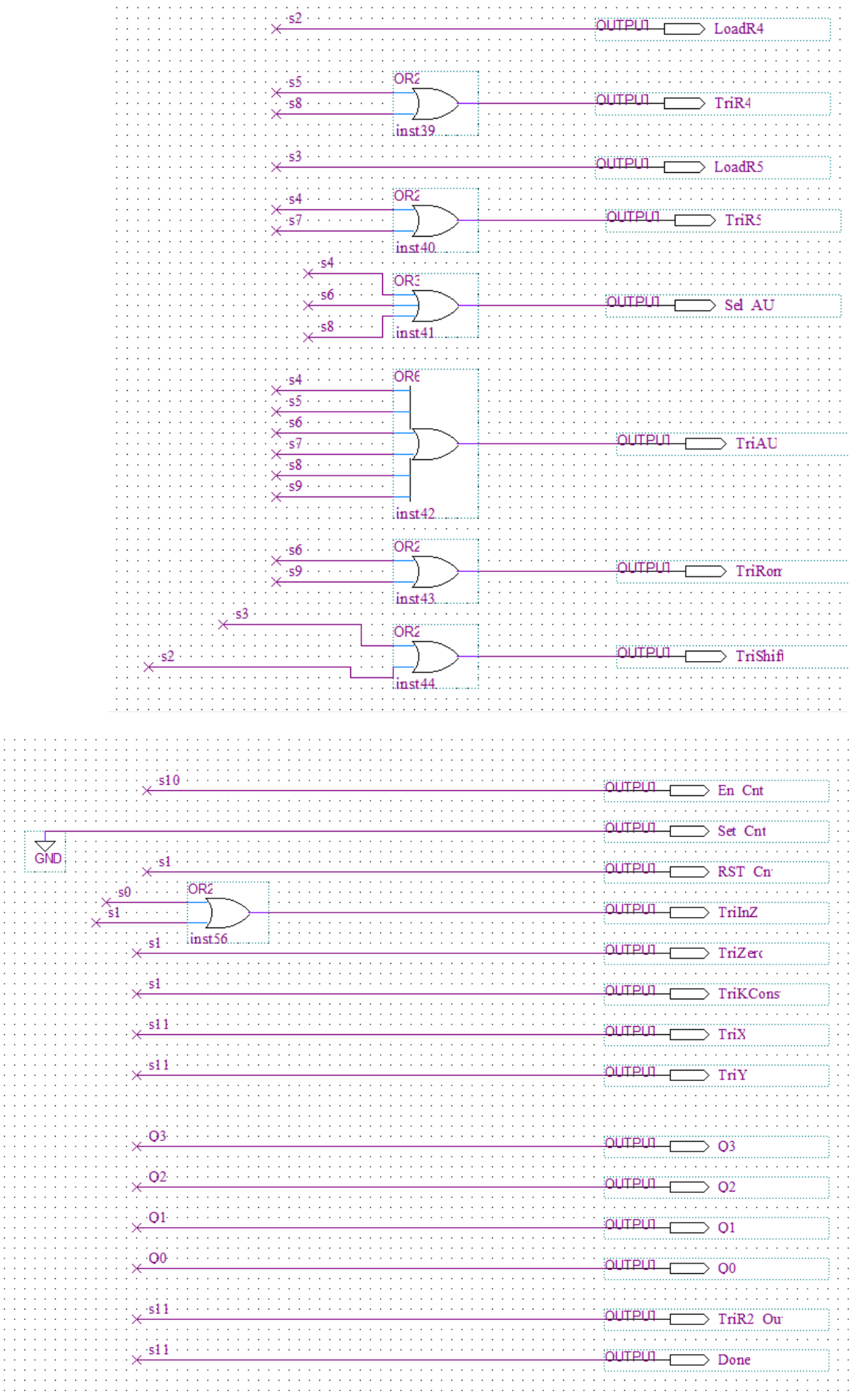
TriZero = s_1

TriKconst = s_1

TriX = s_{11}

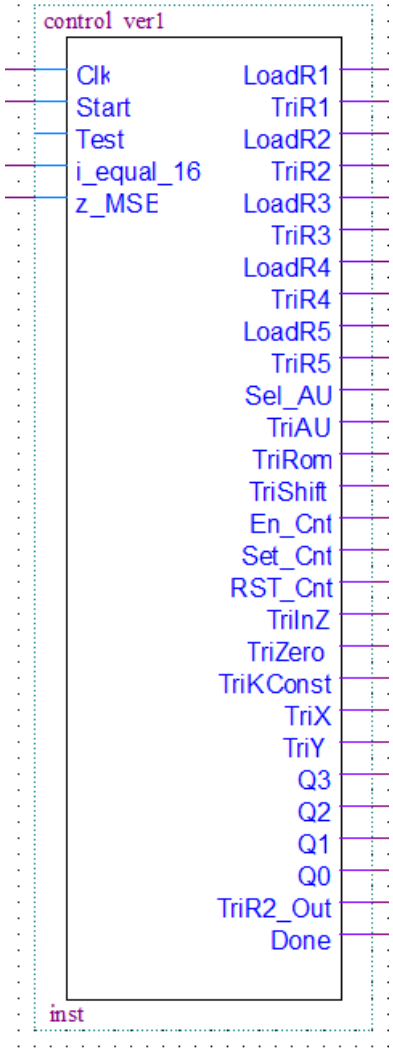
TriY = s_{11}





Hình 3.21. Vẽ mạch cho các tín hiệu điều khiển của khối Controller

`



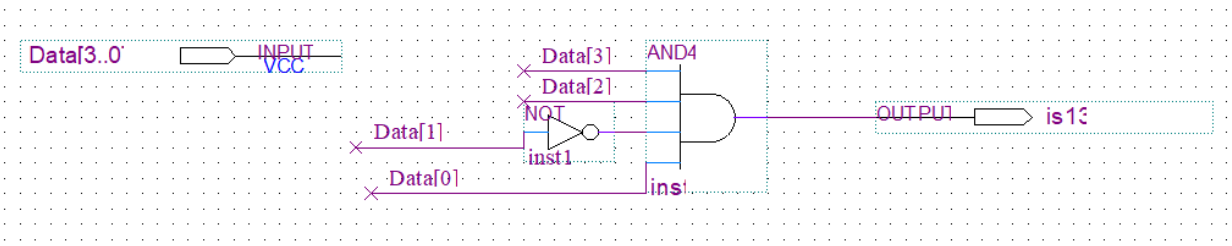
Hình 3.22. Đóng gói khối Controller

3.8. Hoàn thiện mạch ASIC

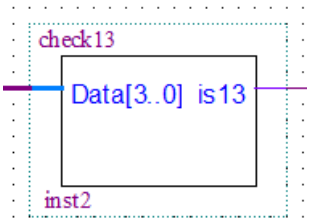
3.8.1. Mạch so sánh điều kiện dừng

Sau khi hoàn tất thiết kế Datapath và Controller, ta cần xây dựng các tín hiệu phản hồi (feedback signals) từ Datapath làm đầu vào cho khối Controller nhằm để quyết định chuyển trạng thái trong FSM.

Ta cần thiết kế một mạch so sánh giữa giá trị của bộ đếm i hiện tại và số lần lặp là 14 bắt đầu từ 0 đến 13. Mạch so sánh sẽ sinh tín hiệu báo hiệu thuật toán đã thực hiện xong vòng lặp, tín hiệu này là i_equal_13 đưa vào khối Controller như một tín hiệu đầu vào. Theo như sơ đồ ASM thì với mỗi vòng lặp i ta sẽ thực hiện tính toán với giá trị i trước, sau đó mới kiểm tra giá trị i để dừng vòng lặp, nên điều kiện dừng sẽ kiểm tra với giá trị 13.

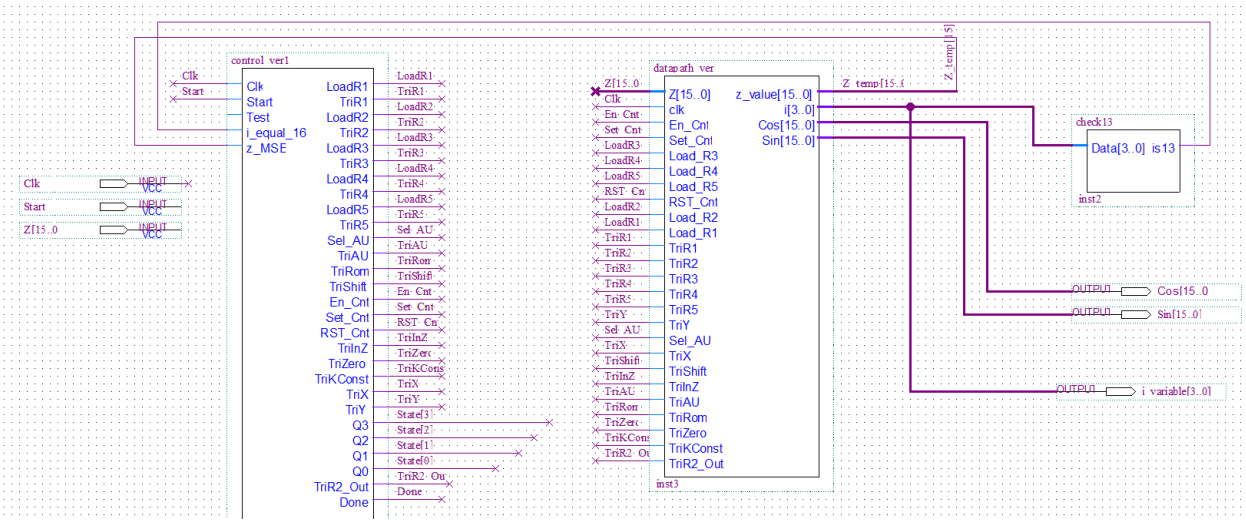


Hình 3.23. Vẽ mạch so sánh với giá trị 13

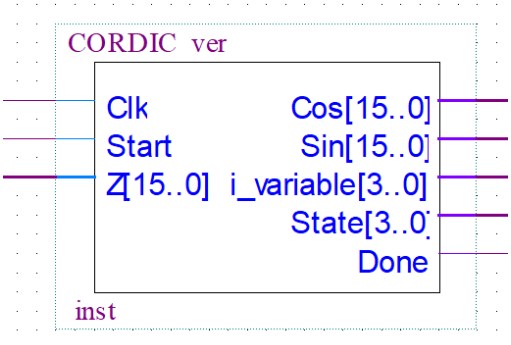


Hình 3.24. Đóng gói khối so sánh với giá trị 13

3.8.2. Hoàn thiện mạch ASIC



Hình 3.25. Hoàn thiện mạch ASIC tính sin và cos của góc ban đầu

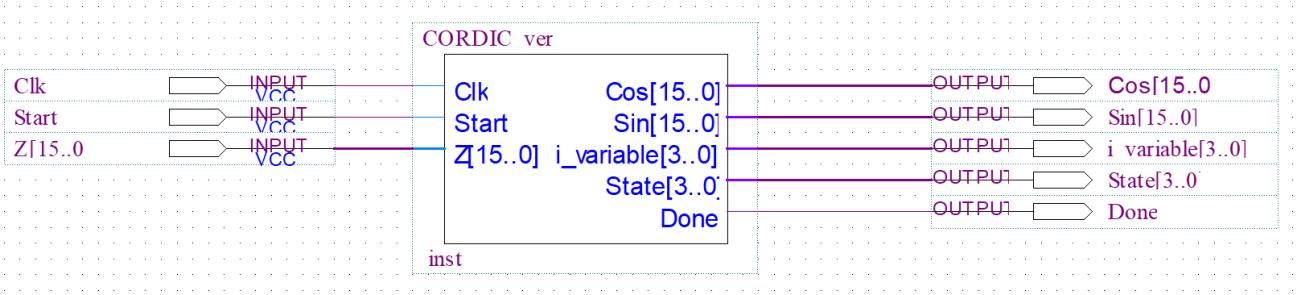


Hình 3.26. Đóng gói mạch ASIC tính sin và cos của góc ban đầu

`

3.9. Mô phỏng mạch trên Quartus

Sau khi đã đóng gói mạch, tiến hành mô phỏng kết quả chạy của mạch trên Quartus.

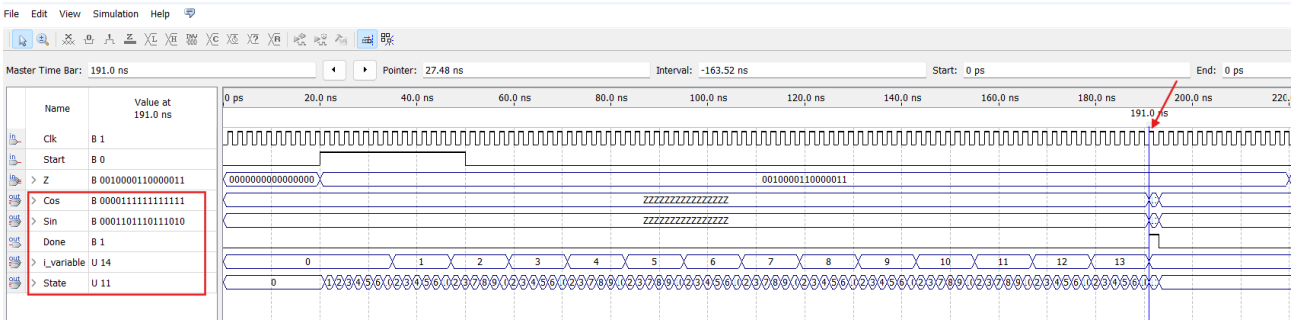


Hình 3.27. Mạch ASIC tính toán sin và cos

Ngoài 3 ngõ ra là kết quả của giá trị sin, cos và tín hiệu Done, để dễ dàng theo dõi ta thêm 2 ngõ ra gồm:

- i_variable: giá trị của biến i (bộ đếm) ở chu kỳ hiện tại của mạch
- State: giá trị trạng thái của mạch ở chu kỳ hiện tại

Tiến hành chạy mô phỏng với góc z cần tính sin và cos là góc $\frac{\pi}{3}$. Kết quả mong muốn thu được là $\sin\frac{\pi}{3} = 0.866025$ và $\cos\frac{\pi}{3} = 0.5$.

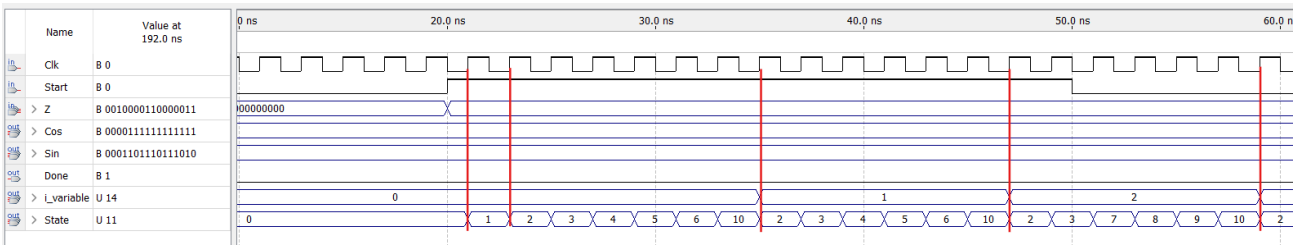


Hình 3.28. Kết quả mô phỏng trên Quartus

Sau khi chạy mô phỏng, sau 14 lần lặp, tại trạng thái s_{11} ta thu được kết quả sin và cos với góc $z = \frac{\pi}{3}$ có giá trị ở dạng số fixed point Q2.13 là 001.000011000011:

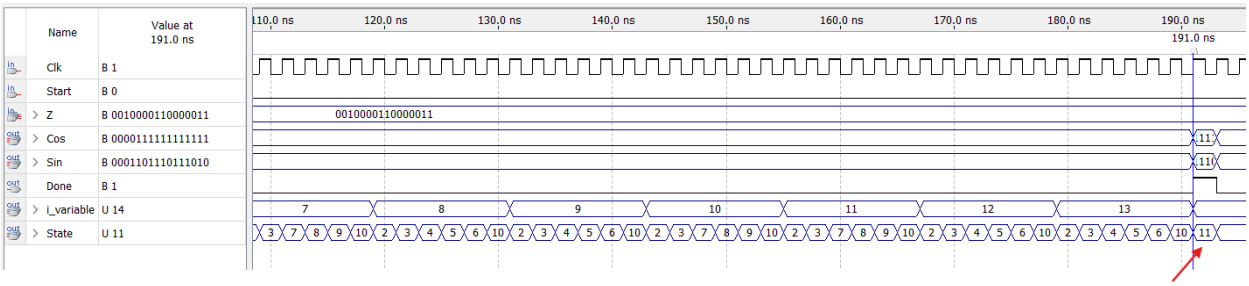
- $\cos z = 000.0111111111111_{Q3.13} \approx 0.49987792 \approx 0.5$
- $\sin z = 000.1101110111010_{Q3.13} \approx 0.86645508$

Như vậy mạch đã thực hiện tính đúng giá trị sin và cos của góc z nhập vào.



Ta nhận xét số chu kì cần thiết cho mạch thực hiện khi tính toán:

- Chu kì thứ 1: Mạch nhận tín hiệu Start
- Chu kì thứ 2: Mạch nhập giá trị đầu vào của góc z và khởi tạo các giá trị
- Chu kì thứ 3 đến chu kì thứ 86: Mạch thực hiện 14 vòng lặp, mỗi vòng lặp cần 6 chu kỳ để xử lý. Vậy cần tốn 84 chu kỳ cho việc thực hiện các vòng lặp này.



- Chu kì thứ 87: Mạch xuất kết quả sin và cos tính được và bật tín hiệu Done.

Như vậy với mỗi input góc z cần tính mạch cần 87 chu kì để tính toán xong giá trị sin và cos.

3.10. Đánh giá chi phí và hiệu suất

3.10.1. Chi phí (Cost)

Sau khi đã tối ưu mạch bằng kỹ thuật Bus Sharing và Register Sharing thì chi phí tổng quát của khối Datapath cần cho mạch gồm:

- 5 thanh ghi lưu trữ 16 bit
- 1 bộ đếm 4 bit
- 1 khối AU tính toán cộng trừ 16 bit
- 1 khối ROM kích thước 14x16 bit
- 1 khối dịch phải tối đa dịch 16 bit
- 16 khối Tri-State 16 bit

3.10.2. Tần số Clock tối đa (F max)

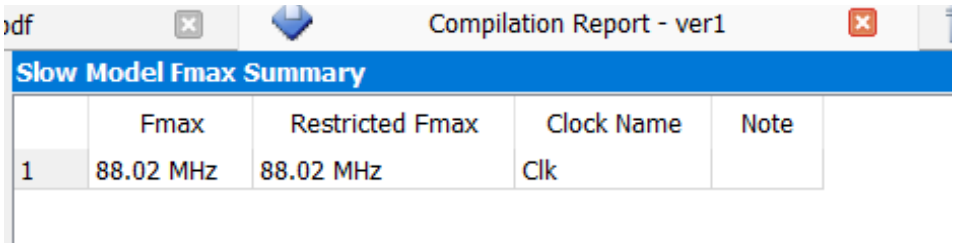
Tần số clock tối đa (F max) của mạch được quyết định bởi độ trễ tổ hợp lớn nhất của các trạng thái trong sơ đồ ASM. Ở mỗi trạng thái, dữ liệu phải được xử lý và ổn định hoàn toàn trong

`

khoảng thời gian giữa hai cạnh xung clock liên tiếp, tức là sau khi tín hiệu clock kích hoạt, dữ liệu đầu ra của khối logic tổ hợp phải sẵn sàng trước khi chuyển sang trạng thái tiếp theo ở cạnh clock kế tiếp.

Do đó, chu kỳ clock tối thiểu của hệ thống phải đủ lớn để bao phủ độ trễ lan truyền lớn nhất (critical path) của đường dữ liệu tương ứng với trạng thái chậm. Trạng thái chậm nhất này hình thành đường critical path và trực tiếp làm giới hạn tần số clock tối đa mà mạch có thể hoạt động ổn định.

Sử dụng phân tích của mô phỏng Quartus, ta thu được giá trị Fmax của mạch:



Hình 3.29. Fmax của mạch trên mô phỏng Quartus

3.10.3. Số chu kỳ thực thi

Từ sơ đồ ASM, số chu kỳ cần cho mạch tính toán giá trị sin và cos với mỗi input z nhập vào là:

- 1 chu kỳ dùng để nhận tín hiệu Start
- 1 chu kỳ dùng để nhập giá trị input z và khởi tạo
- 84 chu kỳ cho 14 vòng lặp tính toán, mỗi vòng lặp cần 6 chu kỳ
- 1 chu kỳ dùng để xuất kết quả và thông báo Done.

Như vậy với mỗi input ta cần 87 chu kỳ cho việc tính toán.

CHƯƠNG 4. TỐI ƯU MẠCH VÀ ĐÁNH GIÁ

4.1. Phân tích đường đi lớn nhất (Critical Path)

Qua phân tích đường đi critical path của mạch, có thể nhận thấy rằng các trạng thái thực hiện tính toán số học đóng vai trò quyết định đến chu kỳ clock của toàn bộ mạch. Cụ thể trong các trạng thái tính toán cộng trừ từ s_4 đến s_6 hoặc từ s_7 đến s_9, sử dụng khối AU thực hiện các phép toán bằng cách cộng các bit qua các Full Adder bằng Ripple-Carry Adder (vì Full Adder sau cần

`

bit Carry Out của Full Adder trước sau khi tính toán xong), tổng độ trễ của khối bằng tổng các độ trễ của 16 Full Adder, dẫn đến khối tính toán AU này có độ trễ lan truyền lớn hơn so với các khối khác trong mạch.

Trạng thái	s_2	s_3	s_4/s_7	s_5/s_8	s_6/s_9	…
Hành động	>>	>>	-/+	+/-	-/+	

Hình 4.1. Thời gian hoạt động của các khối chức năng ở các trạng thái

Do chu kỳ Clock phải được thiết kế sao cho lớn hơn độ trễ của khối AU, nên trong cùng một chu kỳ, các khối còn lại như khối dịch phải ở trạng thái s_2, s_3, các trạng thái nhập, tăng đếm, … chỉ tiêu tốn thời gian rất nhỏ để thực hiện xong (phần màu vàng được tô đậm ở bảng trên). Phần thời gian còn lại trong các chu kỳ đó không được làm gì và ở trạng thái chờ xung Clock tiếp theo.

Kết quả là toàn bộ các trạng thái của mạch, kể cả những trạng thái có những yêu cầu xử lý đơn giản đều phải theo chu kỳ Clock được xác định bởi độ trễ lớn nhất của khối AU, làm tăng tổng thời gian thực thi của toàn mạch. Như vậy độ trễ của khối AU làm giới hạn xung Clock.

4.2. Tối ưu mạch bằng pipeline khối chức năng

4.2.1. Multi-cycling và pipeline 2 tầng khối AU

Để giảm thời gian cần thiết cho một chu kỳ clock, ta cần rút ngắn đường đi critical path của khối AU trong một chu kỳ. Cụ thể thay vì thực hiện toàn bộ các phép toán của khối AU trong một chu kỳ clock du nhất, ta sẽ chia nhỏ quá trình này và phân bố thực hiện qua nhiều chu kỳ clock liên tiếp. Kỹ thuật này được gọi là Multi-cycling, bằng cách tách phép toán và chèn thêm các phần tử nhớ trung gian, khối AU được chuyển từ cấu trúc thực thi đơn chu kỳ sang cấu trúc thực thi đa chu kỳ (multi-cycle).

Giả sử ta cần thực hiện thực 2 phép toán bằng duy nhất 1 khối AU (16 bit) ở 2 chu kỳ liên tiếp, độ trễ của 1 Full Adder là 2ns. Ta phân tích đường đi critical path dưới đây:

	Phép tính 1				**Phép tính 2**			
Chu kì Clock	**1**				**2**			
Khối AU	FA0	FA1	…	FA15	FA0	FA1	…	FA15

Hình 4.2. Đường đi critical path khi thực hiện 2 phép tính trên 1 khối AU đơn chu kì

`

Với mỗi phép tính, ta thực thi trong 1 chu kỳ, dữ liệu phải đi qua 16 Full Adder, và tốn 32ns cho việc thực hiện 1 phép tính. Chu kỳ Clock của mạch phải lớn hơn độ trễ này, tức lớn hơn 32ns. Và thời gian thực hiện 2 phép toán này là 64ns.

Thay vì phải thực hiện tính toán phép tính này trong 1 chu kỳ (đi qua 16 Full Adder), thì ta sẽ thực hiện phép toán này trong 2 chu kỳ:

	Phép tính 1						**Phép tính 2**					
Chu kì Clock	**1**			**2**			**3**			**4**		
Khối AU	FA0	…	FA7	FA8	…	FA15	FA0	…	FA7	FA8	…	FA15

Hình 4.3. Multicycling khối AU

Ở chu kỳ thứ nhất của phép toán 1, ta chỉ cần thực hiện tính toán với 8 Full Adder đầu, sau đó ở chu kỳ thứ hai của phép toán 2, ta tiếp tục lấy kết quả sau khi đã tính toán 8 Full Adder đầu để tính toán tiếp 8 Full Adder sau. Như vậy, chu kỳ Clock của mạch lúc này chỉ cần lớn hơn độ trễ của việc tính toán 8 Full Adder là 16ns. Tuy nhiên số chu kỳ Clock kỳ để thực hiện 1 phép toán lại tăng lên là 2 chu kỳ. Và thời gian thực thi của 2 phép toán vẫn là 64ns. Đây được gọi là Multicycling.

Sau khi đã thực hiện kỹ thuật Multi-Cycling, mỗi phép tính sẽ được thực thi trong 2 chu kỳ. Đối với phép toán thứ nhất, trong chu kỳ clock 1, khối AU sử dụng 8 Full Adder đầu (từ FA0 đến FA7) để thực hiện tính toán cho 8 bit thấp. Sang chu kỳ clock thứ hai, khối AU chỉ sử dụng 8 Full Adder còn lại (từ FA8 đến FA15) để hoàn tất việc tính toán cho 8 bit cao của phép toán 1. Tại thời điểm này (chu kì clock 2), 8 Full Adder đầu tiên (FA0 đến FA7) không còn tham gia vào phép toán 1 và rơi vào trạng thái nhàn rỗi. Các phần tử phần cứng này có thể được tận dụng để thực hiện song song phép toán thứ hai. Ta chia khối AU thành 2 tầng:

		Phép tính 1								
					Phép tính 2					
Chu kì		***i***			***i* + 1**			***i* + 2**		
Khối AU	Tầng 1	FA0	…	FA7	FA0	…	FA7			
	Tầng 2				FA8	…	FA15	FA8	…	FA15

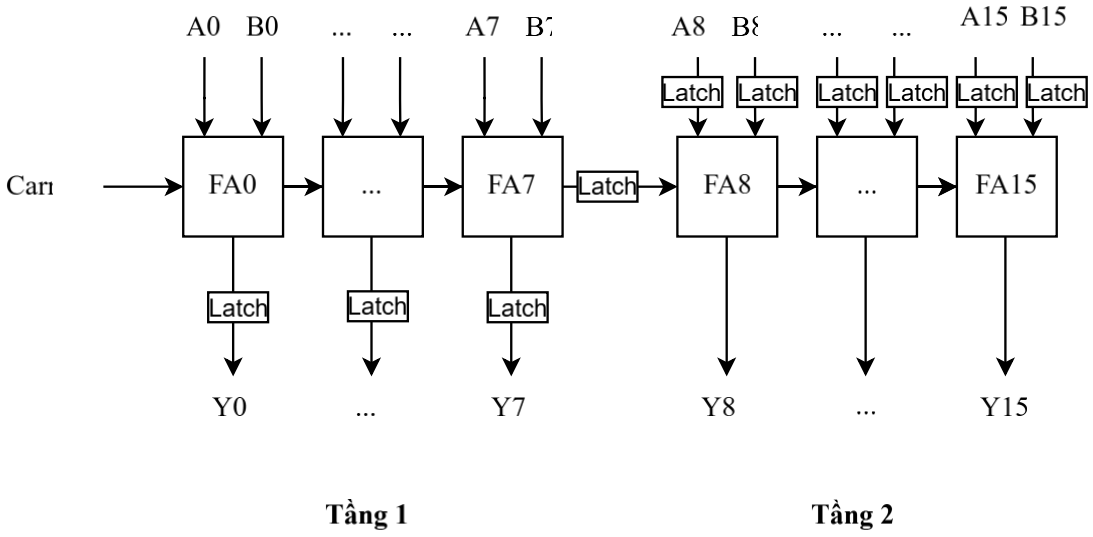
Hình 4.4. Multicycling và Pipeline 2 tầng khối AU

`

Như vậy ta đã thiết kế được khối AU pipeline 2 tầng. Chu kỳ clock của toàn mạch vẫn giống với Multi-Cycling là 16ns, tuy nhiên số chu kỳ cần thực hiện 2 phép toán liên tiếp rút gọn chỉ còn 3 chu kỳ.

4.2.2. Thiết kế khối AU 2 tầng

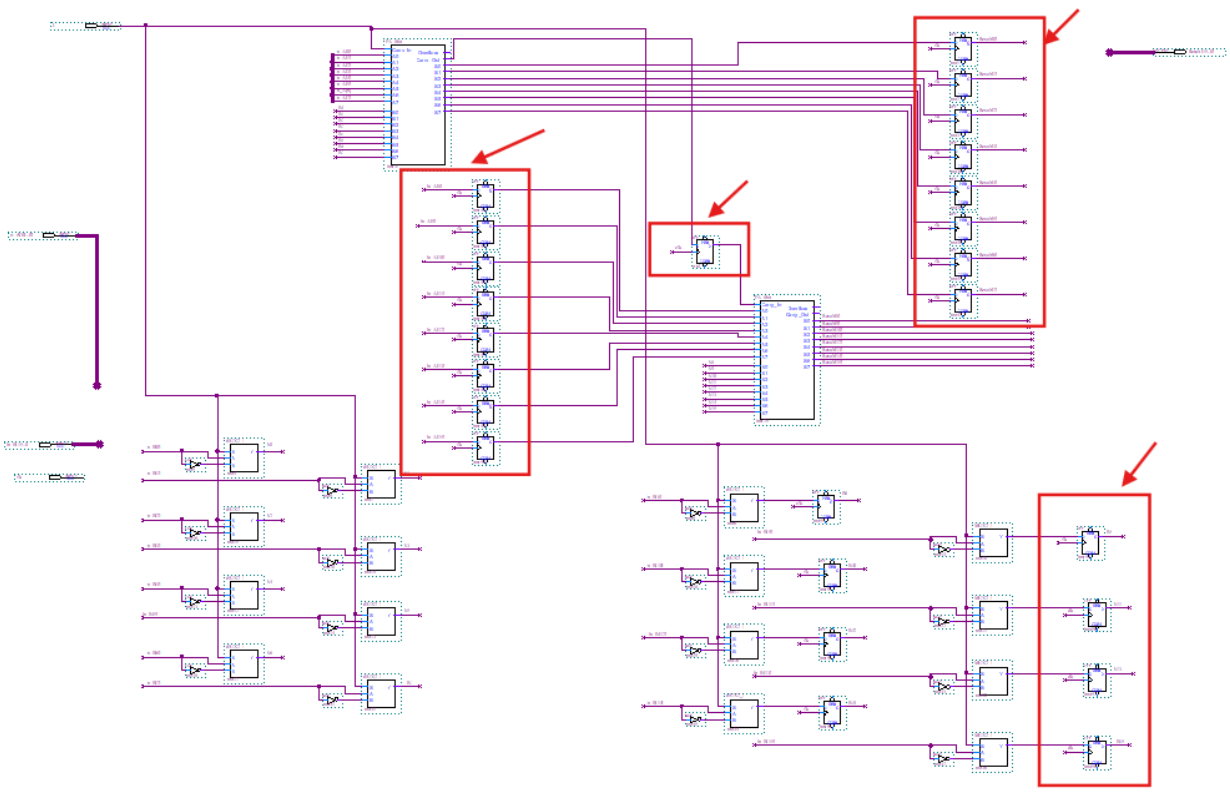
Như đã trình bằng ở trên, ta sẽ thiết kế khối pipeline AU 2 tầng trong đó tầng 1 gồm 8 Full Adder thực hiện tính toán giá trị 8 bit LSB của 2 toán hạng ở chu kỳ thứ nhất và tầng 2 gồm 8 Full Adder thực hiện tính toán 8 bit MSB của 2 toán hạng từ kết quả của bit Carry Out thứ 8:



Hình 4.5. Thiết kế pipeline 2 tầng khối AU

Tuy nhiên khi thực hiện tính toán song song (pipeline) cho nhiều phép toán, ta cần các thiết bị lưu trữ (latch hoặc Flip Flop) để lưu trữ các dữ liệu cần thiết của phép toán trước khi nhập giá trị các toán hạng của phép toán sau.

Vẽ mạch trên Quartus khối AU 2 tầng:



Hình 4.6. Thêm các Flip Flop ở giữa các vị trí khi pipeline 2 tầng khối AU

4.2.3. Thiết kế máy trạng thái FSM cho mạch sau khi pipeline AU 2 tầng

Ở sơ đồ ASM được thiết kế ở chương 3, mạch sẽ thực hiện 3 phép toán liên tiếp với 3 chu kỳ cùng 1 khối AU. Tương tự, sau khi ta đã cải tiến thành khối AU 2 tầng, ta có thể thực hiện pipeline 3 phép toán liên tiếp trong vòng 4 chu kỳ mà không xảy ra đụng độ dữ liệu (tức phép toán sau không cần kết quả tính của phép toán trước). Ở chu kì đầu tiên trong 4 chu kỳ này chỉ thực hiện tính toán ở tầng 1 khối AU mà không thực hiện việc xử lí nào. Sau đó ở 3 chu kì sau sẽ tương ứng với 3 trạng thái thực hiện phép cộng trừ ở sơ đồ ASM ở chương 3.

Ta có bảng thời gian như sau:

Hành động	s_0	s_1	s_2	s_3	s_4/s_8	s_5/s_9	s_6/s_{10}	s_7/s_{11}	s_{12}	s_{13}
Đọc R1			x		x					x
Đọc R2				y		y	z			y
Đọc R3										
Đọc R4						t_1				
Đọc R5					t_2					
AU tầng 1					- / +	+ / -	- / +			

`

AU tầng 2						- / +	+ / -	- / +		
Shift Right			>> i	>> i						
romArctan							rom[i]			
Ghi R1		0.607				x				
Ghi R2		0					y			
Ghi R3		z						z		
Ghi R4			t_1							
Ghi R5				t_2						
Bộ đếm i		0							+1	

Bảng 4.1. Bảng thời gian thực thi của mạch với khối AU 2 tầng

Từ bảng thời gian và sơ đồ ASM ta có được bảng chuyển trạng thái FSM:

Trạng thái hiện tại					**Trạng thái kế tiếp**					
Tên	Q_3	Q_2	Q_1	Q_0	Điều kiện	Tên	Q_3^+	Q_2^+	Q_1^+	Q_0^+
s_0	0	0	0	0	Start = 0	s_0	0	0	0	0
					Start = 1	s_1	0	0	0	1
s_1	0	0	0	1		s_2	0	0	1	0
s_2	0	0	1	0		s_3	0	0	1	1
s_3	0	0	1	1	$z_{MSB} = 0$	s_4	0	1	0	0
					$z_{MSB} = 1$	s_8	1	0	0	0
s_4	0	1	0	0		s_5	0	1	0	1
s_5	0	1	0	1		s_6	0	1	1	0
s_6	0	1	1	0		s_7	0	0	1	1
s_7	0	1	1	1		s_{12}	1	1	0	0
s_8	1	0	0	0		s_9	1	0	0	1
s_9	1	0	0	1		s_{10}	1	0	1	0

`

s_{10}	1	0	1	0		s_{11}	1	0	1	1
s_{11}	1	0	1	1		s_{12}	1	1	0	0
s_{12}	1	1	0	0	i = 16	s_{13}	1	1	0	1
					i ≠ 16	s_2	0	0	1	0
s_{13}	1	1	0	1		s_0	0	0	0	0

Bảng 4.2. Bảng chuyển trạng thái khi kết hợp khối AU 2 tầng

Từ bảng chuyển trạng thái trên ta thu được các phương trình trạng thái tiếp theo như sau:

$$D_3 = Q_3^+ = s_3.(z_{MSB} = 0) + s_7 + s_8 + s_9 + s_{10} + s_{11} + s_{12}.(i = 16)$$

$$D_2 = Q_2^+ = s_3 + s_4 + s_5 + s_6 + s_7 + s_{11} + s_{12}.(i = 16)$$

$$D_1 = Q_1^+ = s_1 + s_2 + s_5 + s_6 + s_9 + s_{10} + s_{12}.(i = 16)'$$

$$D_0 = Q_0^+ = s_0.(Start = 1) + s_2 + s_4 + s_6 + s_8 + s_{10} + s_{12}.(i = 16)$$

4.2.4. Thiết kế các tín hiệu điều khiển cho mạch sau khi pipeline AU 2 tầng

Từ bảng thời gian trên ta lập bảng chân trị các tín hiệu điều khiển:

	Ngõ ra	s_0	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9	s_{10}	s_{11}	s_{12}	s_{13}
R1	LoadR1	x	1	0	0	0	1	0	0	0	1	0	0	0	0
	TriR1	x	0	1	0	1	0	0	0	1	0	0	0	0	1
R2	LoadR2	x	1	0	0	0	0	1	0	0	0	1	0	0	0
	TriR2	x	0	0	1	0	1	0	0	0	1	0	0	0	0
R3	LoadR3	x	1	0	0	0	0	0	1	0	0	0	1	0	0
	TriR3	x	0	0	0	0	0	1	0	0	0	1	0	0	0
R4	LoadR4	x	x	1	0	0	0	0	x	0	0	0	x	x	x
	TriR4	x	0	0	0	0	1	0	0	0	1	0	0	0	0
R5	LoadR5	x	x	0	1	0	0	0	x	0	0	0	x	x	x
	TriR5	x	0	0	0	1	0	0	0	1	0	0	0	0	0

`

AU	Sel_AU	x	x	x	x	1	0	1	0	0	1	0	0	x	x
	TriAU	x	0	0	0	0	1	1	1	0	1	1	1	0	0
ROM	TriRom	x	0	0	0	0	0	1	0	0	0	1	0	0	0
Shift Right	TriShift	x	0	1	1	0	0	0	0	0	0	0	0	0	0
Up Counter	En_Cnt	x	0	0	0	0	0	0	0	0	0	0	0	1	0
	Set_Cnt	x	0	0	0	0	0	0	0	0	0	0	0	0	0
	RST_Cnt	x	1	0	0	0	0	0	0	0	0	0	0	0	0
	TriR2_Out	0	0	0	0	0	0	0	0	0	0	0	0	0	1
	TriInZ	0	1	0	0	0	0	0	0	0	0	0	0	0	0
	TriZero	x	1	0	0	0	0	0	0	0	0	0	0	0	0
	TriKconst	x	1	0	0	0	0	0	0	0	0	0	0	0	0
	TriX	0	0	0	0	0	0	0	0	0	0	0	0	0	1
	TriY	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Bảng 4.3. Bảng sự thật các tín hiệu điều khiển của khối Controller khi áp dụng AU 2 tầng

Phương trình của các ngõ ra của khối Controller:

LoadR1 = $s_1 + s_4 + s_7$

TriR1 = $s_2 + s_4 + s_7 + s_{11}$

LoadR2 = $s_1 + s_5 + s_8$

TriR2 = $s_1 + s_3 + s_5 + s_8$

LoadR3 = $s_1 + s_6 + s_9$

TriR3 = $s_6 + s_9$

LoadR4 = s_2

TriR4 = $s_5 + s_8$

LoadR5 = s_3

TriR5 = $s_4 + s_7$

TriRom = $s_6 + s_9$

TriShift = $s_2 + s_3$

En_Cnt = s_{10}

Set_Cnt = 0

RST_Cnt = s_1

TriR2_Out = s_{13}

TriInZ = s_1

TriZero = s_1

TriKconst = s_1

TriX = s_{13}

`

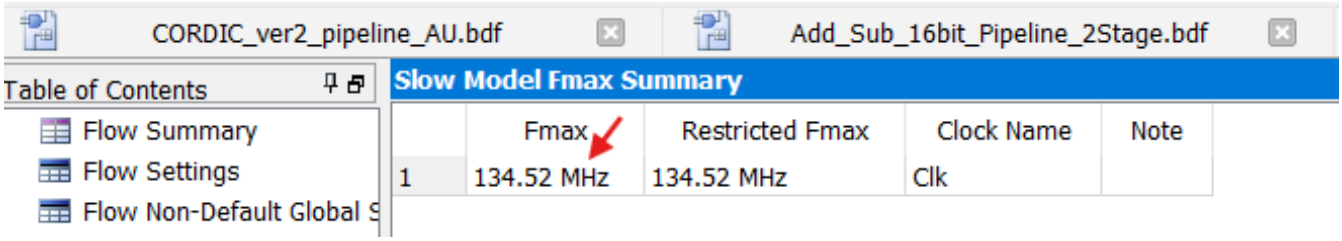
Sel_AU = $s_4 + s_6 + s_8$ TriY = s_{13}

TriAU = $s_4 + s_5 + s_6 + s_7 + s_8 + s_9$

4.2.5. Đánh giá hiệu suất và so sánh mạch sau khi pipeline AU 2 tầng

Sau khi thực hiện pipeline AU 2 tầng, cơ bản thì chi phí không tăng đáng kể chỉ sử dụng thêm 25 thiết bị lưu trữ (Latch hoặc Flip Flop) cho khối AU.

Ta thực hiện chạy mô phỏng Quartus để xem giá trị Fmax sau khi tiến hành tối ưu AU 2 tầng:



Hình 4.7. Fmax của mạch sau khi áp dụng AU 2 tầng

Như vậy Fmax đã tăng từ 88MHz lên 134.5 MHz sau khi đã tối ưu pipeline khối AU 2 tầng.

Ta nhận thấy rằng mỗi lần lặp i ta cần thêm 1 trạng thái để cho tầng 1 khối AU tính toán. Như vậy tổng số chu kì hoàn thành cho mạch sau khi tối ưu là: 87 + 14 = 101 chu kỳ

Bảng so sánh hiệu suất trước và sau khi tối ưu pipeline khối AU 2 tầng:

	Số chu kỳ thực thi	**Fmax (MHz)**
Trước	87	88
Sau	101	134.5

Bảng 4.4. So sánh hiệu suất trước và sau khi tối ưu pipeline khối AU 2 tầng

Hiệu suất sau khi tối ưu:

$$\frac{Tổng\ thời\ gian\ thời\ thi\ mạch\ trước}{Tổng\ thời\ gian\ thực\ thi\ mạch\ sau} = \frac{87 \times \frac{1}{88}}{101 \times \frac{1}{134.5}} = 131{,}655\ \%$$

Như vậy sau khi tối ưu, hiệu suất của mạch tăng thêm 31,655%.

`

4.3. Tối ưu mạch bằng pipeline datapath và AU 2 tầng

4.3.1. Pipeline Datapath 2 tầng

Tương tự như kỹ thuật multi-cycle và pipeline áp dụng cho khối AU quá trình tính toán sin và cos theo thuật toán CORDIC cũng có thể được tối ưu bằng tư tưởng pipeline. Đối với mỗi giá trị góc đầu vào z, ta cần thực hiện 14 vòng lặp liên tiếp để thu được kết quả sin và cos tương ứng. Mỗi input cần 101 chu kì để thực hiện tính toán. Như vậy với n giá trị input góc z ta cần phải 101 x n chu kì thì mới có toàn bộ kết quả.

Dựa trên tư tưởng pipeline, quá trình tính toán được phân chia thành hai tầng xử lý. Tầng thứ nhất thực hiện 7 vòng lặp đầu tiên của thuật toán, tương ứng với các vòng lặp từ 0 đến 6. Tầng thứ hai thực hiện 7 vòng lặp còn lại từ 7 đến 13. Sau khi phân chia, trong khi tầng thứ hai đang xử lý các vòng lặp cuối của giá trị góc đầu vào thứ i, tầng thứ nhất có thể đồng thời tiếp nhận và xử lý giá trị góc đầu vào thứ i + 1, thực hiện song song các vòng lặp đầu tiên.

4.3.2. Sơ đồ pipeline Datapath 2 tầng và AU 2 tầng

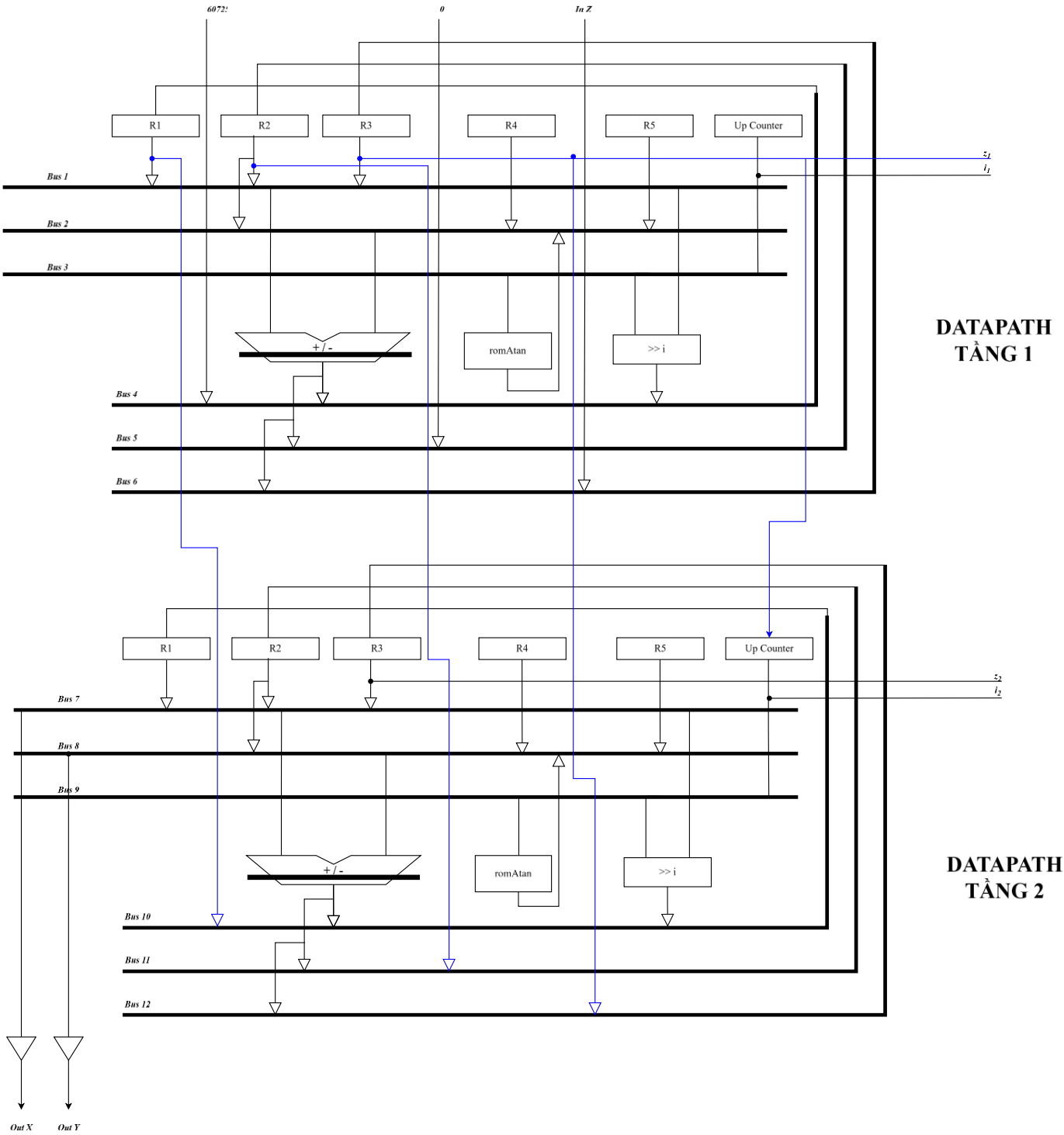
Input i + 1

Input i

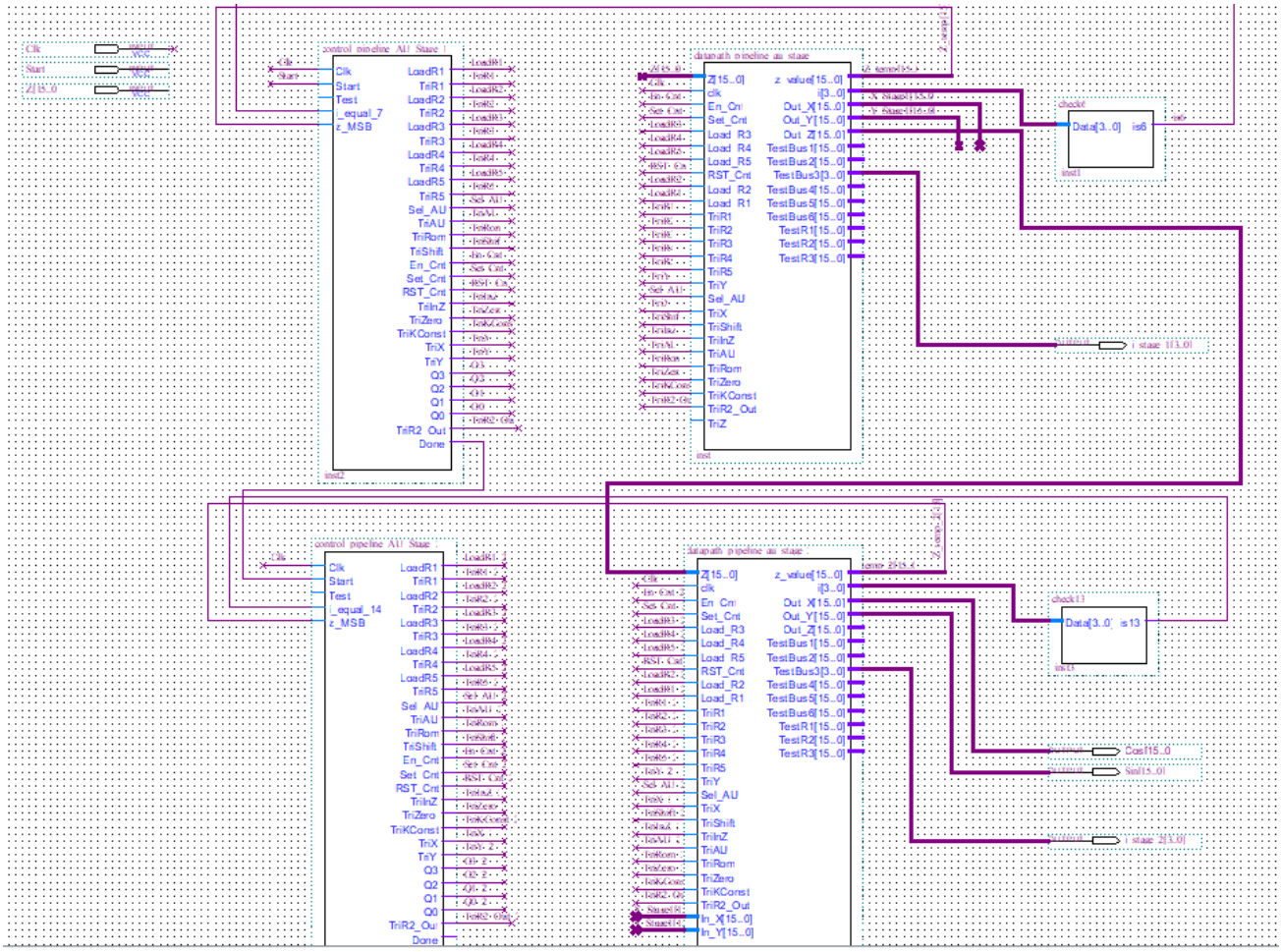
Chu kì	**1**	**2**	**3**	…	**51**	**52**	**53**	**…**	**101**	**…**	**103**	**104**	**105**	**…**	**153**	**…**
Hành động tầng 1		**In1**	7 lần lặp đầu					**In2**	7 lần lặp đầu					**In3**		
Datapath tầng 1	s_0	s_1	s_2	…	s_{12}	$\boldsymbol{s_{13}}$	s_0	s_1	s_2	…	…	$\boldsymbol{s_{13}}$	s_0	s_1	s_2	…
Hành động tầng 2							7 lần lặp sau			**Out1**			7 lần lặp sau			**Out2**
Datapath tầng 2					s_0	$\boldsymbol{s_1}$	s_2	…	s_{12}	s_{13}	s_0	$\boldsymbol{s_1}$	s_2	…	s_{12}	s_{13}

Bảng 4.5. Sơ đồ thời gian của mạch khi áp dụng pipeline 2 tầng Datapath kết hợp với khối AU 2 tầng

Ta nhận thấy rằng các vòng lặp i đều giống nhau về xử lý tại các trạng thái. Ta có thể sử dụng 2 bộ Controller và 2 bộ Datapath được thiết kế ở phần 4.2 kết nối với nhau. Sau khi tầng 1 tính toán xong 7 lần lặp đầu tiên, các giá trị x, y, z và giá trị đếm i sẽ được nạp xuống ở tầng 2 tại trạng thái Output tầng 1 (s_{13} ở tầng 1) và tại trạng thái nhập giá trị Input của tầng 2 (s_1 ở tầng 2). Tín hiệu Done sau khi xử lí xong của tầng 1 sẽ là tín hiệu Start của tầng 2.



Hình 4.8. Datapath sau khi thực hiện pipeline Datapath 2 tầng và khối AU 2 tầng



Hình 4.9. Vẽ mạch sau khi cải tiến pipeline Datapath 2 tầng và khối AU 2 tầng

4.4. Đánh giá thông số các mạch

	Số chu kỳ thực thi 1 input	Số chu kỳ thực thi n input liên tiếp	Fmax (MHz)	Chi phí
Mạch không pipeline	87	87 x n	88	1 AU, 1 ROM, 1 Shift 5 Register, 1 Counter
Mạch pipeline AU 2 tầng	101	101 x n	134.5	1 AU, 1 ROM, 1 Shift 5 Register, 1 Counter 25 (Flip Flop hoặc Latch)
Mạch pipeline Datapath 2 tầng và AU 2 tầng	102	102 + (52 x n)	134.5	Gấp 2 lần chi phí mạch pipeline AU 2 tầng

Bảng 4.6. Bảng so sánh trước và sau khi áp dụng các kỹ thuật tối ưu pipeline khối AU 2 tầng và Datapath 2 tầng.

`

CHƯƠNG 5. HIỆN THỰC MẠCH TRÊN KIT DE2

5.1. Hiện thị số fixed-point 16 bit trên LED 7 đoạn

Để hiện thị giá trị số ở định dạng fixed-point 16 bit Q2.13 trên LED 7 đoạn, ta cần chuyển đổi phần thập phân của số sang dạng phù hợp để hiển thị từng chữ số. Như ở đây ta muốn hiển thị giá trị phần nguyên và 4 số sau dấu thập phân, ta sẽ tiến hành nhân giá trị fixed-point hiện tại với hệ số 10000 nhằm dịch chuyển 4 chữ số phần thập phân sang phần nguyên. Sau đó dịch 13 bit tương ứng với việc chia cho 2^{13} để bù lại hệ số tỷ lệ của định dạng Q2.13.

Lúc này, phần giá trị thu được ở hệ 10 là 1 số nguyên bao gồm 5 chữ số, gồm 1 chữ số của phần nguyên ban đầu và 4 chữ số sau dấu phẩy ban đầu ở hệ 10. Giá trị nhị phân này gồm 17 bit (1 bit dấu và 16 bit giá trị) ở dạng bù 2. Để hiển thị một giá trị nhị phân lên LED 7 đoạn ta cần chuyển giá trị nhị phân này thành giá trị BCD.

Để hiện thị BCD ta sử dụng thuật toán Double Dabble để chuyển đổi giá trị một số nguyên dương ở dạng nhị phân sang BCD. Với giá trị đã tính ở trên vẫn còn ở dạng bù 2, ta sẽ lấy giá trị tuyệt đối của số này sau đó sử dụng 1 LED hiển thị dấu bằng bit dấu giá trị trên, và 5 LED hiển thị giá trị tuyệt đối của số nguyên này.

5.2. Thuật toán Double Dabble

Ta cần hiện thị 1 số 16 bit với giá trị thập phân lên đến $2^{16} = 65536$ gồm 5 chữ số thập phân. Giá trị BCD tương ứng gồm 20 bit với mỗi chữ số thập phân cần 4 bit BCD.

Input đưa vào là 1 số nhị phân 16 bit

Output ngõ ra là 1 số BCD 20 bit.

Thuật toán Double Dabble dùng để chuyển đổi mã BCD theo thuật toán như sau:

```
Bin[15..0] = Giá trị nhị phân 16 bit
BCD[19..0] = Khởi tạo giá trị BCD 20 bit là 0
while (i khác 16){
   if (BCD[3..0] >= 5) BCD[3..0] = BCD[3..0] + 3;
   if (BCD[7..4] >= 5) BCD[7..4] = BCD[7..4] + 3;
```

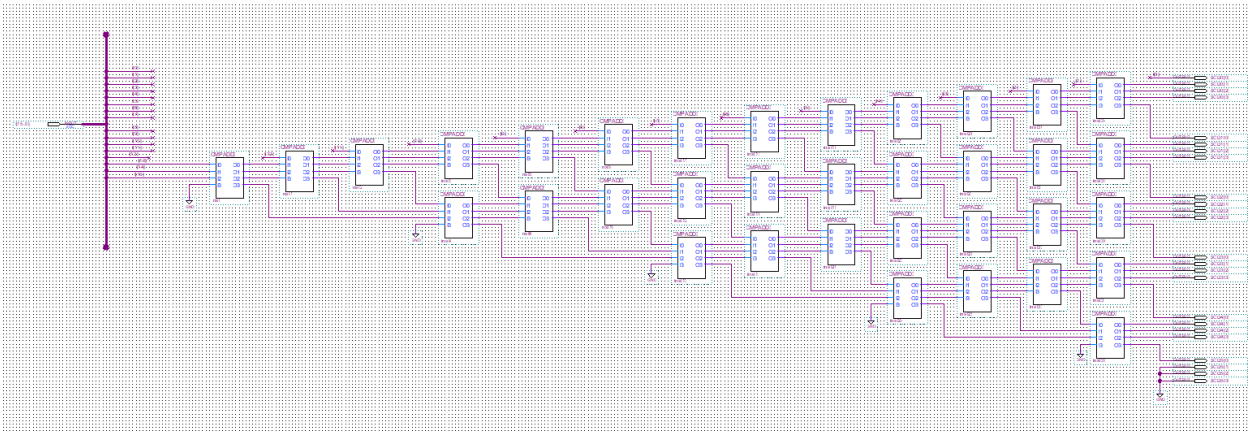
`

```
    if (BCD[11..8] >= 5) BCD[11..8] = BCD[11..8] + 3;
    if (BCD[15..12] >= 5) BCD[15..12] = BCD[15..12] + 3;
    if (BCD[19..16] >= 5) BCD[19..16]  = BCD[19..16] + 3;
    BCD = BCD << 1;
    BCD[0] = Bin[15];
    Bin = Bin << 1;
    i = i + 1;
}
Output = BCD[19..0]
```

Hình 5.1. Thuật toán Double Dabble chuyển đổi số nhị phân 16 bit sang số BCD 20 bit.

Sau khi đã thực hiện chuyển đổi giá trị nhị phân thành giá trị BCD ta sử dụng giá trị BCD này đưa vào các IC 7447 để hiện thị các chữ số trên LED 7 đoạn, cứ 4 bit BCD là 1 chữ số thập phân.

Trong mạch này để tránh phải chờ thêm xung Clock cho thuật toán Double Dabble ta sẽ thực hiện vẽ mạch theo dạng tuần tự:



Hình 5.2. Vẽ mạch tổ hợp theo thuật toán Double Dabble

5.3. Hiện thực kết quả mạch thiết kế trên KIT DE2

5.3.1. Link video nạp mạch ASIC tính sin và cos trên KIT DE2

Link video nạp KIT DE2: https://www.youtube.com/watch?v=UhiPGlQJAAA

`

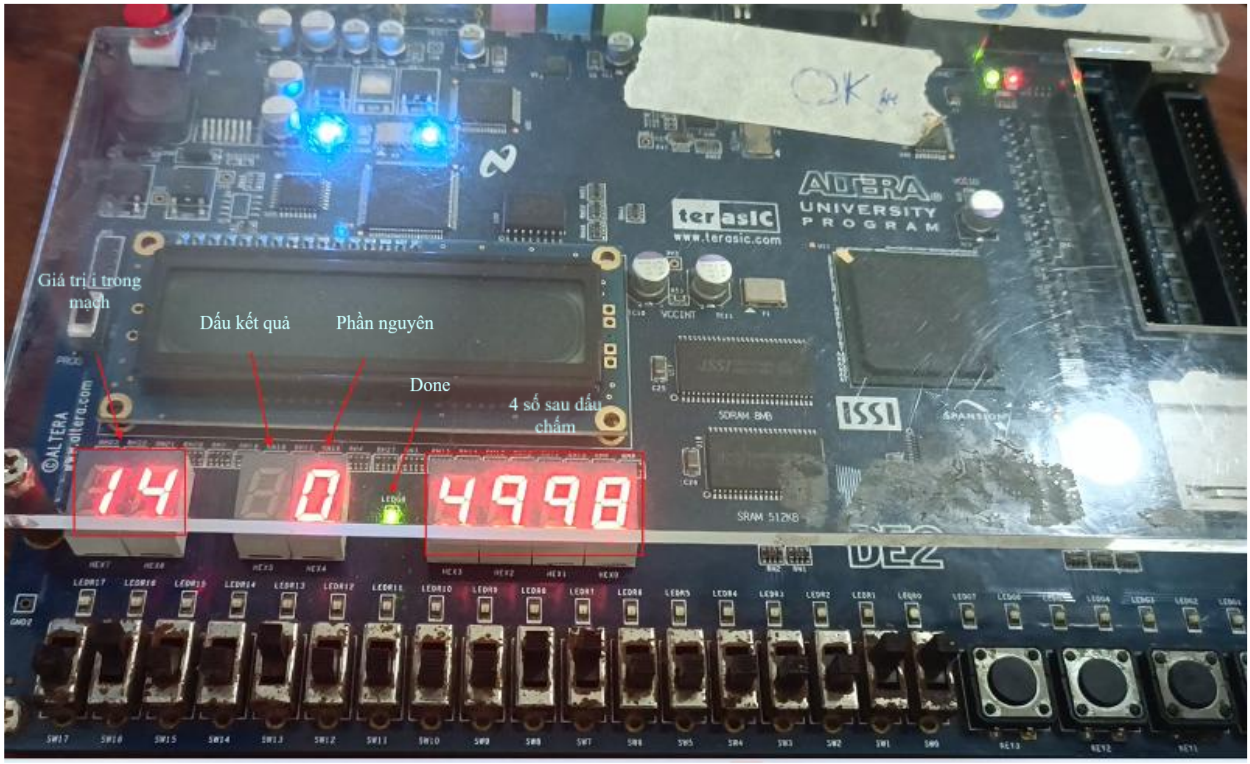
5.3.2. Kịch bản nạp KIT

Input	Giá trị Input góc z	Giá trị fixed point của z	Giá trị cos (hệ 10)	Giá trị sin (hệ 10)
1	$\frac{\pi}{3} = 1.0472$	001 0000110000011	0.5	0.8660
2	$\frac{-\pi}{4} = -0.7854$	111 0011011011110	0.7071	-0.7071

Bảng 5.1. Bảng các giá trị góc z để mô phỏng nạp KIT DE2

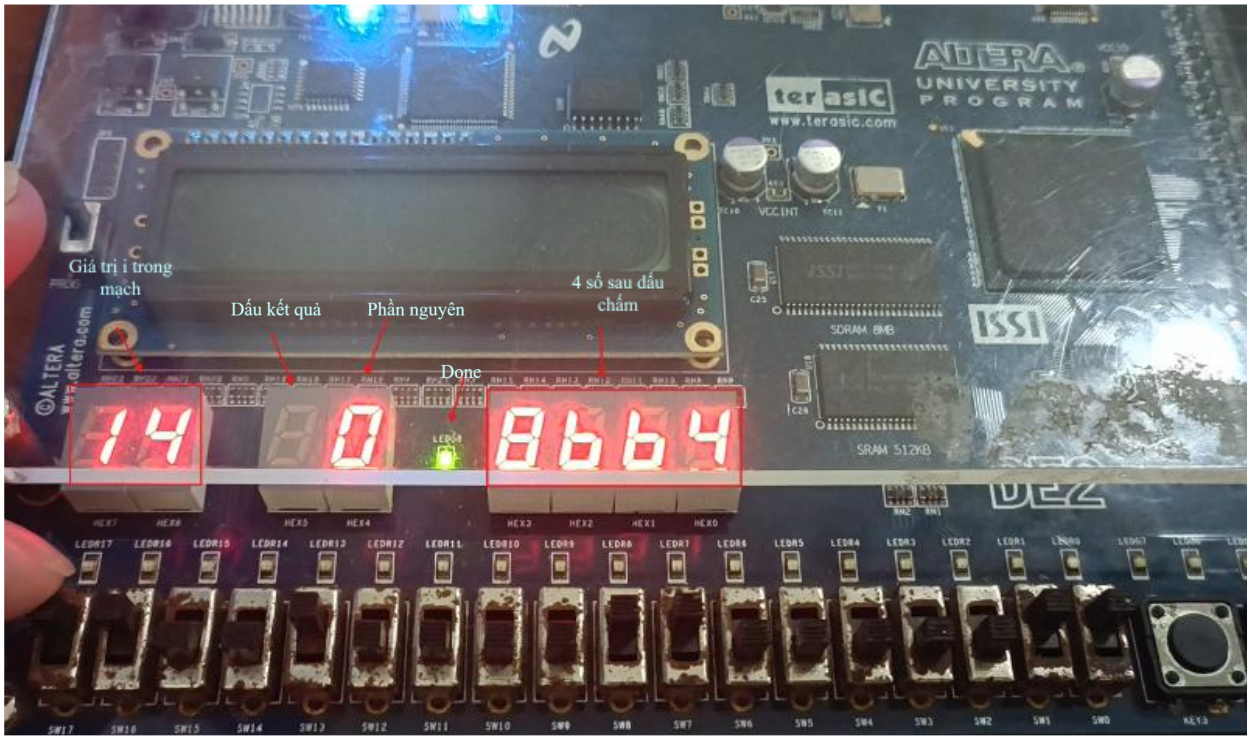
5.3.3. Kết quả nạp mạch không sử dụng pipeline trên KIT DE2

5.3.3.1. Testcase 1



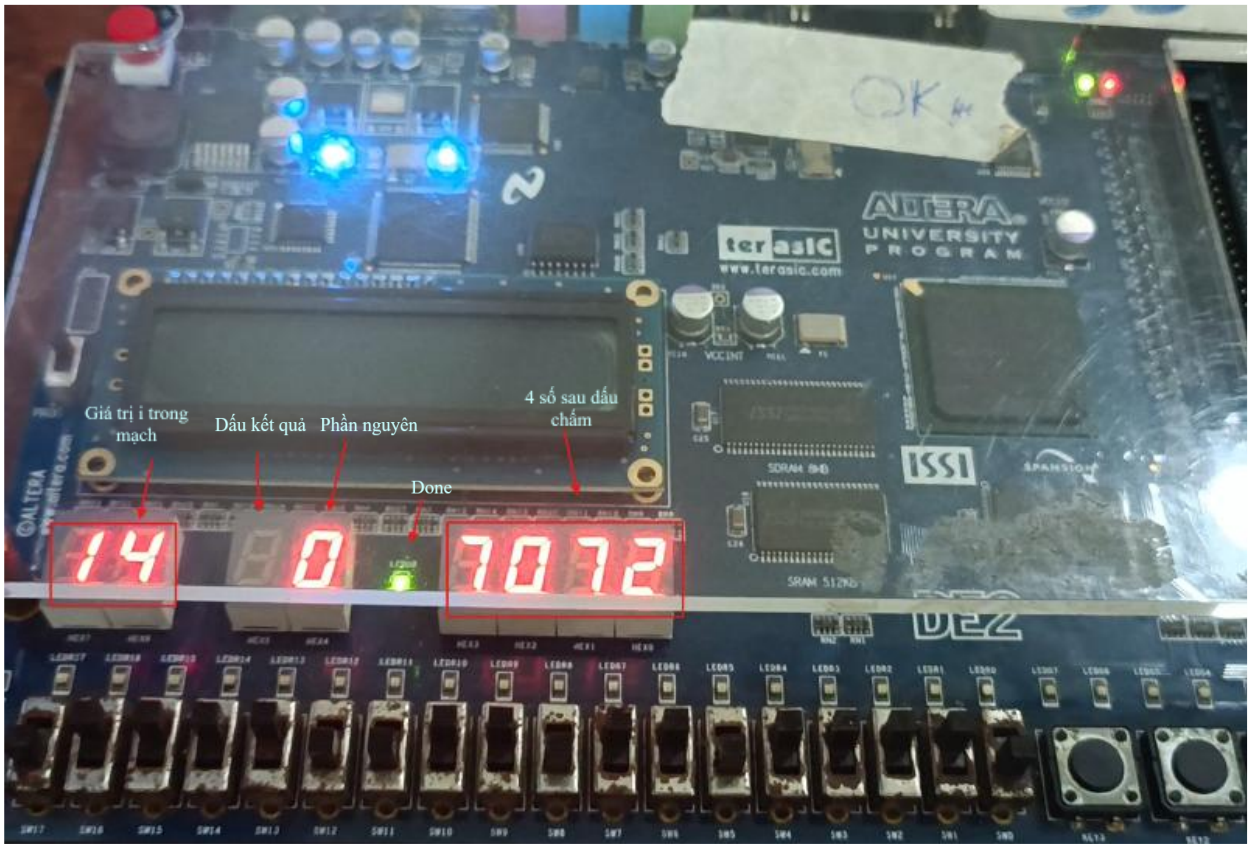
Hình 5.3. Giá trị cos của góc $\frac{\pi}{3}$ trên mô phỏng KIT

`



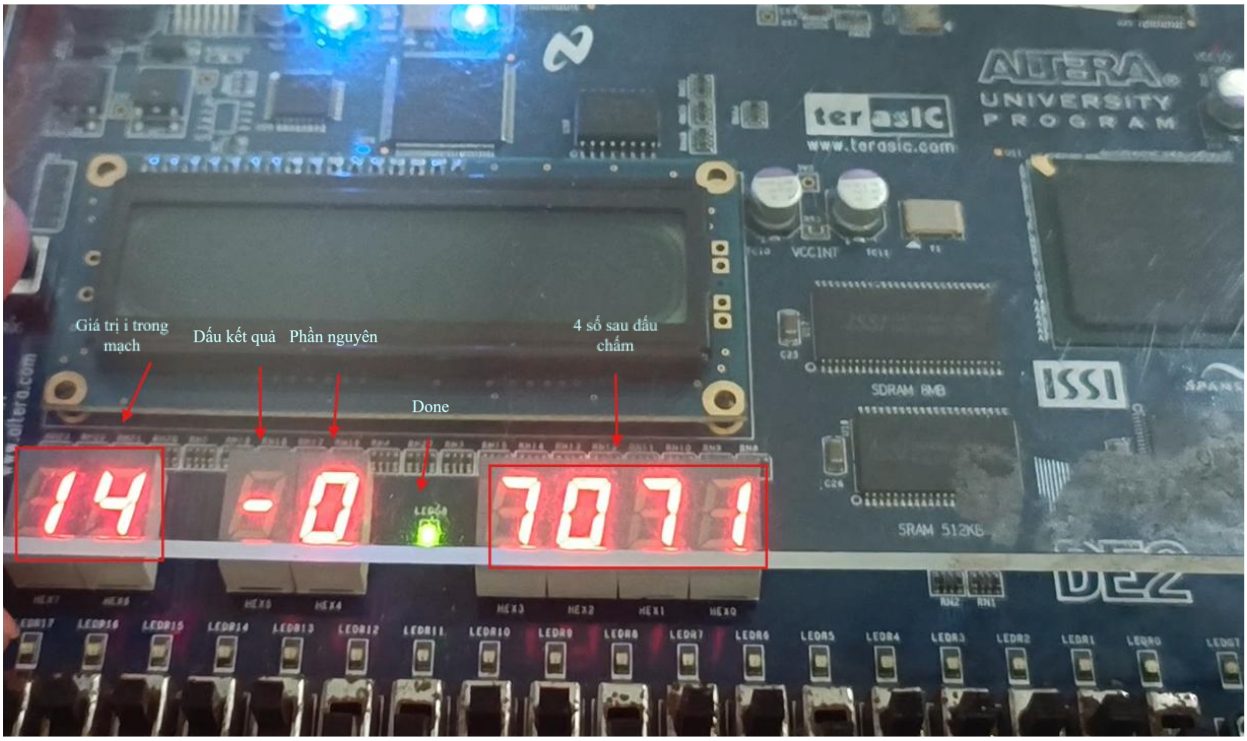
Hình 5.4. Giá trị sin của góc $\frac{\pi}{3}$ trên mô phỏng KIT

5.3.3.2. Testcase 2



Hình 5.5. Giá trị cos của góc $\frac{-\pi}{4}$ trên mô phỏng KIT

`



Hình 5.6. Giá trị sin của góc $\frac{-\pi}{4}$ trên mô phỏng KIT

`